

ASIC Design and Performance Optimization of Vortex: An Open-Source RISC-V GPGPU Architecture

Submitted By:

Abdelhmed Mahmoud (21P0171)

Abdullah Ahmed Youssef (2100369)

Ahmed Mostafa (21P0407)

Amir Raed (21P0380)

Eiz Eldeen Housam (21P0170)

Mark Maher (21P0355)

Sama Ashraf (21P0066)

Youssef Khaled (22P0278)

Under Supervision of:

Dr.DiaaEldin Sayed – Ain Shams University

Dr. Hossam Hassan - Siliconarts

SILICONARTS



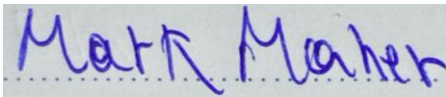
Communications
Graduation Project Report
Dr.DiaaEldin Sayed
Dr. Hossam Hassan
27/1/2026



DECLARATION

We hereby certify that this material, which we now submit for assessment on the programme of study leading to the award of Bachelor of Science in *(insert title of degree for which registered)* is entirely our own work, that we have exercised reasonable care to ensure that the work is original, and does not to the best of our knowledge breach any law of copyright, and have not been taken from the work of others and to the extent that such work has been cited and acknowledged within the text of our work.

Signed: by all students

Abdelhmed Mahmoud	<i>Abdelhameed Mahmoud Sayed</i>
Abdullah Ahmed Youssef	<i>Abdullah Ahmed Youssef</i>
Ahmed Mostafa	<i>A h m e d m o s t a f a G o m a a</i>
Amir Raed	<i>Amir Raed</i>
Eiz Eldeen Housam	<i>Ezz Eldeen Hossam Elhussary</i>
Mark Maher	
Sama Ashraf	<i>Sama Ashraf-Mabrouk</i>
Youssef Khaled	<i>Youssef Khaled Abo Al Fotouh</i> CamScanner - Scanned with

Date: Tuesday, 27th January 2026.

CONTRIBUTIONS

Name	Tasks
Abdelhmed Mahmoud	Researched on the memory system and synthesized the LSU
Abdullah Ahmed Youssef	Researched the memory system and synthesized the LSU
Ahmed Mostafa	Researched the memory system and synthesized the ALU
Amir Raed	Researched the fetch, decode, commit stages and synthesized the SFU
Eiz Eldeen Housam	Researched the execution unit stage and verified the FPU
Mark Maher	Researched the SIMT , schedule stage and synthesized the SFU
Sama Ashraf	Researched the execution unit stage and verified the ALU
Youssef Khaled	Researched the difference between the CPU and GPU and synthesized the ALU

Note: this project is a collaborated by credit and mainstream students together.

ACKNOWLEDGMENT

We would like to express sincere gratitude to **Dr. Hossam Hassan** from Silicon art sand **Dr. DiaaEldin Sayed** from Faculty of Engineering, Ain Shams University, for his invaluable technical support, expert guidance, and the foundational information provided throughout the development of this report. His contributions were instrumental in clarifying the complexities of the RISC-V instruction set extensions and the memory hierarchy optimizations discussed herein.

We would also like to thank **Eng. Abnoub** from Faculty of Engineering, Ain Shams University for providing valuable insights and sharing key reference materials that significantly supported the progress of this work.

ABSTRACT

This report presents the ASIC implementation and physical design of the Vortex RISC-V GPGPU architecture, with particular emphasis on its microarchitectural organization. The work analyzes the SIMT execution model and contrasts it with conventional CPU designs, then details the Vortex core architecture, including its 6-stage warp pipeline, hierarchical warp scheduler, IPDOM-based divergence handling, and scalable core/warp configuration parameters. The report further describes the GPU-oriented memory hierarchy, featuring non-blocking caches and high-bandwidth load/store units, as well as the RISC-V ISA extensions that enable warp management, thread masking, and synchronization. Building on this architectural foundation, the project converts the original FPGA-targeted RTL into an ASIC-ready design, performs full-chip synthesis and backend implementation, and evaluates the resulting chip in terms of power, performance, and area, demonstrating the practicality of realizing Vortex as a hardened RISC-V GPU accelerator.

TABLE OF CONTENTS

LIST OF FIGURES AND TABLES	IV
LIST OF ACRONYMS/ABBREVIATIONS	V
LIST OF UTILIZED STANDARDS	VIII
1 INTRODUCTION.....	1
2 RISC-V GPGPU ARCHITECTURE	2
2.1 RISC-GPU VS CPU: ARCHITECTURAL DIFFERENCES	2
2.1.1 Execution and parallelism model	2
2.1.2 Scheduling and latency hiding.....	3
2.1.3 Control flow and divergence	3
2.1.4 ISA and programmability	3
2.1.5 Memory system emphasis.	4
2.2 SINGLE INSTRUCTION MULTIPLE THREADS	4
2.2.1 SIMD vs SIMT vs SMT	6
2.2.2 Single Instruction, Multiple Data(SIMD).....	7
2.2.3 Single Instruction, Multiple Threads(SIMT).....	8
2.2.4 Programming Model Benefit.....	9
2.2.5 When SIMT Is Most Effective	9
2.3 Schedule stage	9
2.3.1 Program Counter (PC).....	10
2.3.2 Barrier Synchronization	10
2.4 Fetch stage	12
2.4.1 Role and Context	12
2.4.2 Architecture & Data Flow	12
2.4.3 Fetch operation	13
2.4.4 Request Buffering.....	14
2.4.5 Complete Data Flow.....	14
2.5 Decode stage	15
2.5.1 Role & context	15
2.5.2 The Decoding Process	15
2.5.3 Key Responsibilities of decode stage	17
2.6 issue stage	18
2.6.1 Instruction Buffer	18
2.6.2 Scoreboard.....	19
2.6.3 Operands Collector (OPC)	19
2.6.4 Dataflow Overview	19
2.7 excute stage.....	20
2.7.1 Core Responsibilities.....	20
2.7.2 Functional Units	20
2.7.3 Dispatch Mechanism	22
2.7.4 Execution Flow	22
2.7.1 Latency Characteristics	22
2.7.2 Parallel Execution and Warp Distribution.....	23
2.7.3 Key Takeaways	23
2.8 Commit Stage	23
2.8.1 Role & context	23
2.8.2 Inputs & Outputs	24

2.8.3	Arbitration	24
2.8.4	Instruction Retirement Counting	25
2.8.5	Warp Tracking.....	25
2.8.6	Writeback	25
2.9	memory system.....	26
2.9.1	Introduction	26
2.9.2	Memory Hierarchy	26
2.9.3	L1 Cache	28
2.9.4	Cache Bank Architecture.....	31
2.9.5	LSU Slice Architecture	38
2.9.6	Memory Coalescing, Scheduling, and Response Tracking	40
2.10	software stack.....	44
2.10.1	Compilation Flow.....	44
2.10.2	GPU Compilation Flow	46
2.11	Configuration and instruction set	47
2.11.1	Base ISA.....	48
3	ASIC IMPLEMENTATION	50
3.1	FPGA-to-ASIC Transition	50
3.2	SFU	50
3.2.1	SFU Synthesis Methodology.....	50
3.2.2	Top-Level Design Definition and Formality Setup	50
3.2.3	Library and Technology Configuration	50
3.2.4	RTL Analysis and Elaboration	51
3.2.5	Path Grouping and Timing Organization	51
3.2.6	Timing Constraints Definition.....	51
3.2.7	Mapping and Optimization.....	52
3.2.8	Reporting and Analysis	52
3.2.9	Area Utilization Analysis	54
3.3	FPU	55
3.3.1	Design overview.....	55
3.3.2	Synthesis Constraints	55
4	VERIFICATION.....	56
4.1	Hierarchical Verification Strategy.....	56
4.2	CHALLENGES AND AUTOMATION.....	56
4.3	Testbench Architecture and Implementation.....	57
4.3.1	Self-Checking Testbench Architecture.....	57
4.3.2	System-Level Sanity Checks.....	58
4.3.3	OpenCL Verification Flow.....	58
5	VERIFICATION RESULTS.....	59
5.1	FUNCTIONAL VERIFICATION.....	59
5.2	Regression Testing	60
6	CONCLUSION	61
7	FUTURE WORK	62
7.1	Verification.....	62
7.1.1	MILESTONE 1: POST-SYNTHESIS VERIFICATION	62

7.1.2	MILESTONE 2: GATE-LEVEL SIMULATION (GLS).....	62
7.1.3	MILESTONE 3: OPTIMIZATION AND BENCHMARKING.....	62
7.2	ASIC	63
7.2.1	Milestone 1: Full-Chip Synthesis	63
7.2.2	Milestone 2: Baseline Physical Implementation.....	63
7.2.3	Milestone 3: Microarchitectural Optimizations (Opt1 & Opt2)	63
7.2.4	Milestone 4: Signoff and Documentation.....	63
8	ARABIC SUMMARY.....	64
9	REFERENCES.....	65

LIST OF FIGURES AND TABLES

Figure 1, Vortex RISC GPU	2
Figure 2,Single Instruction Multiple Threads	4
Figure 3, SIMT Instructions	5
Figure 4, SIMT Dual Issue GPU Core Design.....	6
Figure 5, Streaming multi-processor unit	8
Figure 6, SIMT Core.....	8
Figure 7, Barrier Synchronization.....	11
Figure 8,Fetch stage architecture	13
Figure 9, Complete fetch data flow diagram.....	15
Figure 10,execution unit selection	16
Table 1,Decode stage outputs	17
Table 2, Latency Characteristics	22
Figure 11, Memory Hierarchy	27
Table 3 ,Comparison with L1 D-Cache	28
Table 4,DRAM Technology Comparison.....	30
Table 5,Comparison with Caches	30
Figure 12,LSU Slice act as Bridge.....	38
Table 6,Compilation Separation	45
Figure 13,Host completion Flow	46
Figure 14,GPU compilation flow.....	47
Table 7, Instruction set of the RISC-V Gpu.....	47
Figure 15,RISC-V Base Instruction Formats	48
Table 8: SFU Setup Timing Summary.....	53
Table 9: SFU Hold Timing Summary.....	53
Table 10: SFU Overall Area Summary.....	54
Table 12, Synthesis Constraints	55
Figure 16, Block Diagram of the Self-Checking Testbench Architecture	57
Figure 17,OpenCL Host Code of the Software-Based Golden Reference Check.....	58
Figure 18,OpenCL Kernel Source Code Executed on the Vortex RTL for System Validation...	59
Figure 19,Functional Simulation Waveforms of the FPU_DSP Unit	59
Figure 20,: Terminal output showing the successful execution of the vecadd regression test.....	60

LIST OF ACRONYMS/ABBREVIATIONS

ACRONYMS	ABBREVIATIONS
GPU	Graphics Processing Unit
ISA	Instruction Set Architecture
GPGPU	General-Purpose Graphics Processing Unit
FPGA	Field-Programmable Gate Array
ASIC	Application-Specific Integrated Circuit
IP	Intellectual Property
RAM	Random Access Memory
DSP	Digital Signal Processing
LUT	Look-Up Table
RTL	Register Transfer Level
SIMT	Single Instruction, Multiple Threads
LSU	Load/Store Unit
FPU	Floating Point Unit
RISC	Reduced Instruction Set Computer
CPU	Central Processing Unit
SIMD	Single Instruction, Multiple Data
SMT	Simultaneous Multithreading
ALU	Arithmetic Logic Unit
PC	Program Counter
SM	Streaming Multiprocessor
TMC	Thread Mask Convergence
ICache	Instruction Cache
RV32	RISC-V 32-bit
RV64	RISC-V 64-bit
UUID	Unique Instruction Identifier
WID	Warp ID
DCache	Data Cache
IBUF	Instruction Buffer
L1	Level 1 Cache
TMASK	Thread Mask
RD	Destination Register
RS	Source Register

SFU	Special Function Unit
TCU	Tensor Core Unit
WB	Writeback
OPC	Operands Collector
RAW	Read After Write
WAR	Write After Read
FU	Functional Unit
XLEN	Register Length
CSR	Control and Status Register
MAC	Multiply-Accumulate
ML	Machine Learning
SOP	Start of Packet
EOP	End of Packet
INSTRET	Instructions Retired
MINSTRET	Machine Instructions Retired
Popcount	Population Count
L2	Level 2 Cache
L3	Level 3 Cache
DRAM	Dynamic Random Access Memory
GDDR	Graphics Double Data Rate
HBM	High Bandwidth Memory
HPC	High Performance Computing
MSHR	Miss Status Holding Registers
SRAM	Static Random Access Memory
LRU	Least Recently Used
OpenCL	Open Computing Language
CUDA	Compute Unified Device Architecture
ELF	Executable and Linkable Format
GCC	GNU Compiler Collection
API	Application Programming Interface
PoCL	Portable Computing Language
IR	Intermediate Representation
LLVM	Low Level Virtual Machine
SVF	Synthesis Verification File
DDC	Design Compiler Database

SDC	Synopsys Design Constraints
SDF	Standard Delay Format
SS	Slow-Slow Corner
TT	Typical-Typical Corner
FF	Fast-Fast Corner
NRE	Non-Recurring Engineering
ECO	Engineering Change Order
INREG	Input-to-Register Paths
REGOUT	Register-to-Output Paths
INOUT	Input-to-Output Paths
FMA	Fused Multiply-Add
LZC	Leading Zero Counter
IEEE	Institute of Electrical and Electronics Engineers
DUT	Design Under Test
LEC	Logical Equivalence Checking
GLS	Gate-Level Simulation
DRC	Design Rule Check
LVS	Layout Versus Schematic
CTS	Clock Tree Synthesis
PPA	Power, Performance, Area
STA	Static Timing Analysis

LIST OF UTILIZED STANDARDS

- **RISC-V ISA base and extensions**

- Implements the standard 32-bit RISC-V base ISA with integer and multiply extensions (RV32IM), and in newer versions RV32IMF/RV32IMAFD for floating-point support.
- Uses a **minimal set of custom GPU/SIMT instructions** added on top of the standard RISC-V ISA, designed to keep compatibility with existing RISC-V toolchains and ABI conventions.

- **RISC-V ABI and calling convention**

- Adheres to the standard RISC-V ABI, including argument passing and return value conventions, so GPU intrinsics and runtime calls integrate cleanly with existing RISC-V software stacks.

- **OpenCL standard**

- Provides a complete software stack compatible with **OpenCL 1.2**, enabling Vortex to run standard OpenCL kernels without requiring a proprietary programming model.

- **PCI Express–based system integration**

- The reference FPGA/SoC platforms expose Vortex as a PCIe-attached accelerator, following conventional PCIe-based GPU attachment used in many heterogeneous systems.

1 INTRODUCTION

Graphics processing units (GPUs) have evolved from fixed-function graphics engines into highly programmable accelerators that deliver massive parallel throughput for general-purpose workloads such as machine learning, scientific computing, and image processing. In parallel, the RISC-V instruction set architecture has emerged as an open, extensible standard that enables academic and industrial communities to explore new processor designs without proprietary constraints. Within this context, the Vortex project proposes an open-source RISC-V-based GPGPU that combines a compact set of SIMT-oriented ISA extensions with a scalable, many-core microarchitecture, making it an attractive platform for research and deployment of programmable accelerators.

This report investigates the ASIC realization of the Vortex RISC-V GPU, with a focus on understanding and implementing its microarchitectural features in a full industrial VLSI flow. The work begins by outlining the architectural differences between conventional scalar CPUs and SIMT-style RISC-V GPUs, then describes the Vortex core organization, including its 6-stage warp pipeline, warp scheduler, divergence management using IPDOM-based control, and GPU-optimized memory hierarchy. Based on the publicly available RTL, the design is adapted from an FPGA-oriented environment to an ASIC-ready implementation, synthesized to a target technology, and taken through floorplanning, place-and-route, and signoff analysis. The objectives of the project are to validate the synthesizability of the Vortex architecture, to quantify its power, performance, and area (PPA) characteristics in silicon, and to document an end-to-end methodology for implementing open-source RISC-V GPUs as hardened accelerators.

2 RISC-V GPGPU ARCHITECTURE

2.1 RISC-GPU VS CPU: ARCHITECTURAL DIFFERENCES

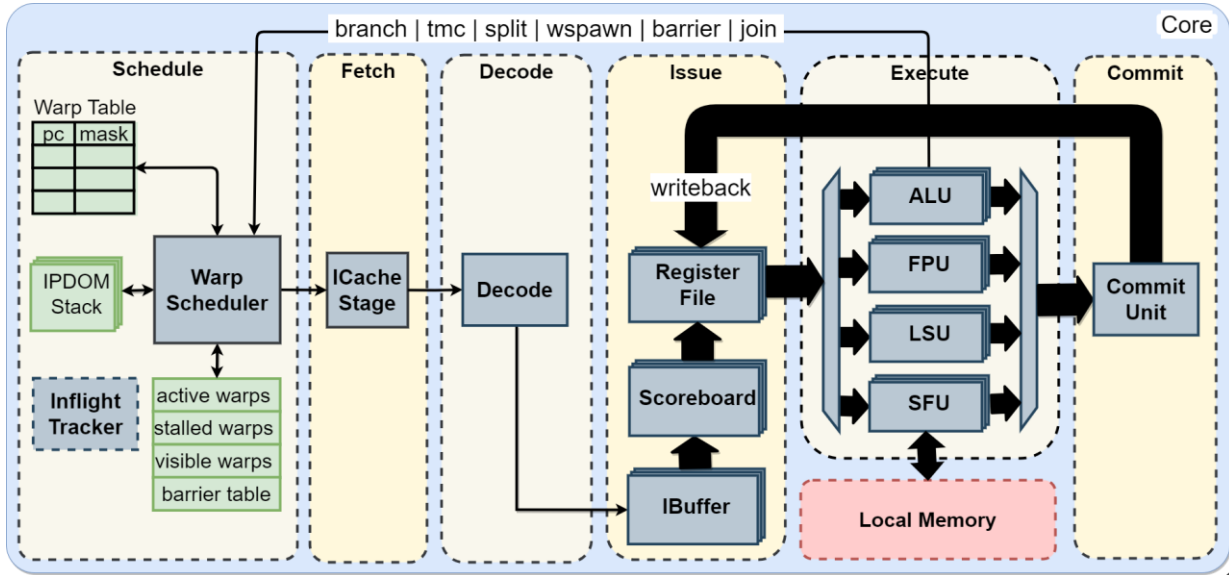


Figure 1, Vortex RISC GPU

A RISC-GPU such as Vortex is designed to maximize **throughput** by executing large numbers of lightweight threads in parallel, while a CPU is designed to minimize latency for a smaller number of threads with strong single-thread performance. This fundamental difference drives distinct choices in execution model, scheduling, control flow handling, and memory hierarchy organization.

2.1.1 Execution and parallelism model

CPUs primarily implement MIMD execution (independent instruction streams per core/thread), often enhanced with out-of-order execution and complex control logic to exploit instruction-level parallelism. In contrast, Vortex follows a SIMT model: threads are grouped into warps that share a common instruction stream, with a per-thread active mask controlling which lanes participate in each instruction.

2.1.2 Scheduling and latency hiding

CPU performance relies heavily on speculation, deep pipelines, and large caches to reduce the impact of unpredictable control flow and memory latency. A RISC-GPU instead hides latency by maintaining many resident warps and rapidly switching execution to a ready warp when others stall, enabling high utilization even under long memory delays.

2.1.3 Control flow and divergence

On a CPU, divergent branches naturally proceed independently because each thread maintains its own program counter and state. In SIMT GPUs, divergence within a warp can reduce efficiency; Vortex addresses this using explicit SIMT control mechanisms (e.g., split/join) and reconvergence tracking (IPDOM stack) to ensure correctness while managing performance impact.

2.1.4 ISA and programmability

Vortex is built on the RISC-V ISA and extends it with a small set of SIMT-oriented instructions (e.g., warp creation, thread activation, divergence control, and barriers) to support GPU execution while retaining a RISC-V base programming model. This differs from typical CPUs, where the ISA is not augmented with warp-level control primitives because the execution model does not require explicit SIMT management.

2.1.5 Memory system emphasis.

CPU memory hierarchies prioritize low-latency access, sophisticated caching, and coherence to support general-purpose workloads with irregular access patterns. GPUs prioritize bandwidth and parallel memory throughput; Vortex similarly emphasizes GPU-oriented memory behavior and synchronization primitives (e.g., bar) that align with throughput execution and many-thread coordination.

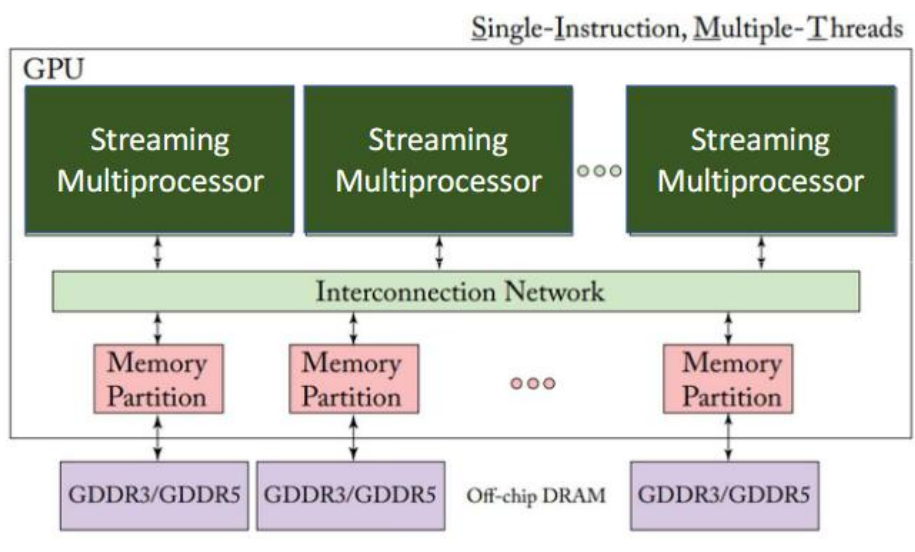


Figure 2, Single Instruction Multiple Threads

2.2 SINGLE INSTRUCTION MULTIPLE THREADS

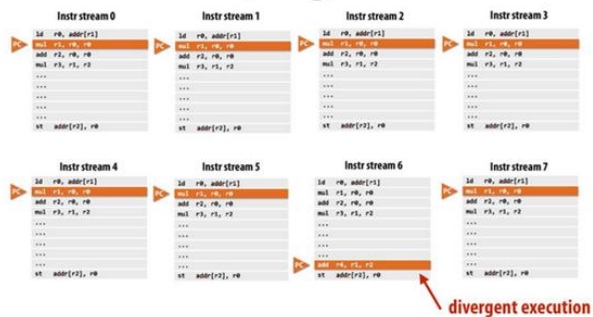
Single Instruction Multiple Threads (SIMT) is the fundamental execution model in modern General-Purpose Computing on Graphics Processing Units (GPGPU), used by NVIDIA and AMD (two major technology companies that design processors—especially **graphics processing units (GPUs)** to manage high-throughput parallelism. It enables thousands of threads to execute the same instruction simultaneously on different data points, while managing individual thread states, branching, and control flow within hardware-managed groups called warps (usually 32–64 threads).

Key Aspects of SIMT in GPGPU:

- **Warp execution:** The GPU executes one instruction for a whole group of threads (a *warp*, usually 32–64 threads), which reduces control overhead.
- **Thread autonomy:** Each thread has its own registers and program flow, making programming easy, even though hardware executes them together like SIMD.
- **Branch divergence handling:** When threads take different branches, the GPU runs each path separately using masks, which can reduce performance if many threads diverge.
- **Latency hiding:** If one warp stalls (e.g., waiting for memory), the GPU quickly switches to another ready warp to keep the hardware busy.

SIMT architecture allows developers to write code for independent threads while the hardware manages the efficient, parallel, grouped execution

GPU “SIMT” (single instruction multiple thread)



Many modern GPUs execute hardware threads that run instruction streams with only scalar instructions.

GPU cores detect when different hardware threads are executing the same instruction, and implement simultaneous execution of up to SIMD-width threads using SIMD ALUs.

Here ALU 6 would be “masked off” since thread 6 is not executing the same instruction as the other hardware threads.

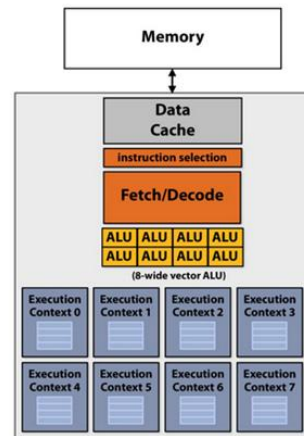


Figure 3, SIMT Instructions

The Execution in the SIMT core

- In each cycle, the hardware selects a warp for scheduling

- The warp's program counter is used to access an instruction memory

to find the next instruction to execute for the warp

- An on-chip warp buffer holds multiple warps

for a GPU SM. (Why ?)

- **Hide memory latency** – when a warp waits for global memory, other warps can execute.
- **Avoid idle execution units** – keeps ALUs, FPU, and other cores busy.

Support thread-level parallelism – GPUs rely on thousands of threads; storing multiple warps on-chip allows fast context switching

SIMT Dual Issue GPU Core Design

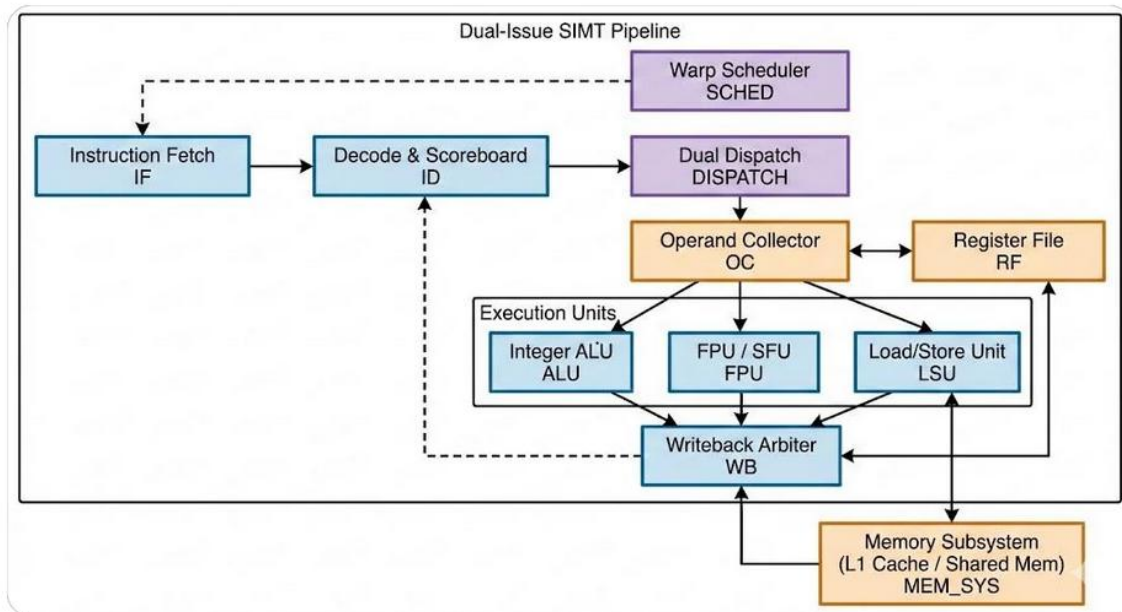


Figure 4, SIMT Dual Issue GPU Core Design

2.2.1 SIMD vs SIMT vs SMT

SIMT and SIMD both approach parallelism through broadcasting the same instruction to multiple execution units. This way, you replicate the execution units, but they all share the same fetch/decode hardware.

- SIMD: HW pipeline width must be known by SW

- SIMT: Pipeline width hidden from SW

If so, what's the difference between "single instruction, multiple *data*", and single instruction, multiple *threads*"? In NVIDIA's model, there are 3 key features that SIMD doesn't have:

1. Single instruction, multiple register sets
2. Single instruction, multiple addresses
3. Single instruction, multiple flow paths

SIMD is a hardware concept

SIMT is a programming + execution model

SIMT is basically:

"SIMD hardware pretending to be many independent threads"

2.2.2 Single Instruction, Multiple Data(SIMD)

SIMD (Single Instruction, Multiple Data) is an execution paradigm that exploits data-level parallelism by applying a single instruction to multiple data elements concurrently using a set of parallel lanes under a shared control path. In a SIMD datapath, the instruction fetch and decode logic are centralized, while the arithmetic units and register file entries are replicated across lanes, allowing one vector instruction to perform several identical operations (for example, additions or multiplications) in a single cycle. This organization significantly improves throughput and energy efficiency for regular workloads such as linear algebra, signal processing, and image manipulation, where the same operation is repeatedly applied over contiguous arrays of data. However, SIMD efficiency degrades in the presence of irregular control flow or memory access patterns; masking and predication can partially address control divergence but may leave some lanes idle, reducing effective utilization.

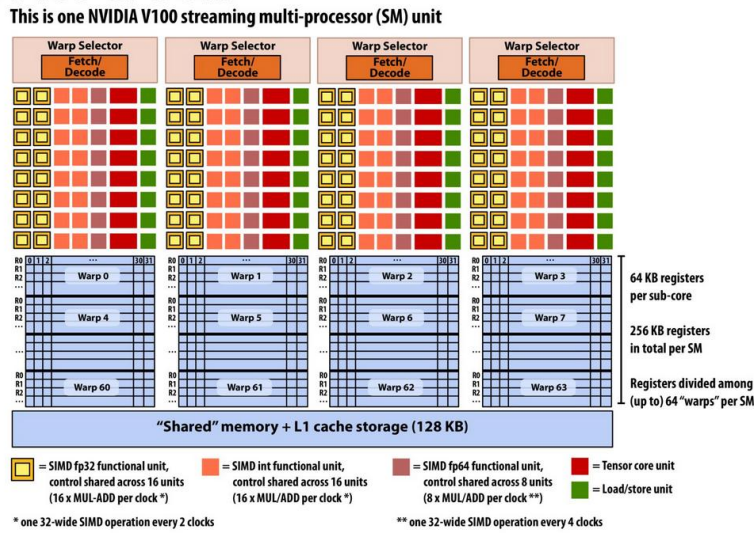


Figure 5, Streaming multi-processor unit

2.2.3 Single Instruction, Multiple Threads(SIMT)

SIMT (Single Instruction, Multiple Threads) is the execution model used by modern GPUs and RISC-V GPGPUs such as Vortex, in which groups of threads (warps) execute a common instruction stream in lockstep while maintaining separate architectural state per thread (registers, program counter, predicates). The hardware presents a thread-centric programming model—each thread appears independent—but internally, threads are organized into fixed-size warps that share a single instruction fetch/decode path and are issued together onto a set of parallel execution lanes.

Within a warp, all active threads execute the same instruction in the same cycle, and per-

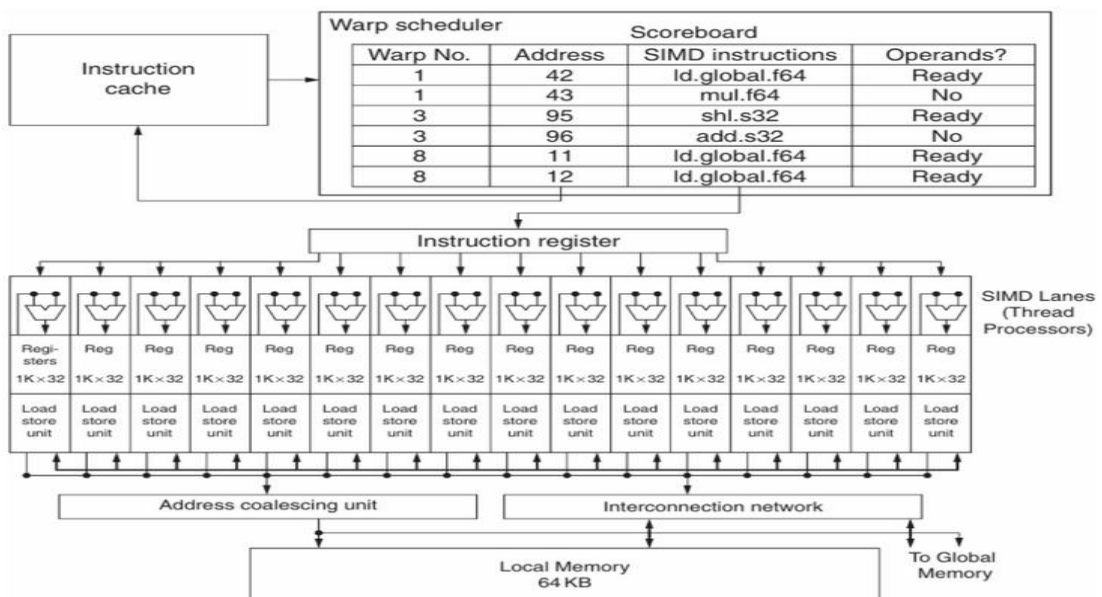


Figure 6, SIMT Core

thread masks determine which lanes are enabled for a particular instruction, allowing the model to tolerate limited control-flow divergence at the cost of reduced lane utilization when threads follow different paths. This organization combines the control-efficiency of SIMD (one control path driving many lanes) with a more flexible MIMD-like programming interface, enabling high throughput and scalable latency hiding for massively parallel workloads such as those targeted by Vortex.

2.2.4 Programming Model Benefit

In traditional SIMD architectures, the programmer (or compiler) must explicitly express vector operations, ensure proper data alignment, and manage mapping of loop iterations onto vector lanes. Under the SIMT model, each thread is expressed as a conventional scalar control flow, and the hardware groups threads into warps and handles their parallel execution transparently, which significantly simplifies the programming of parallel kernels.

2.2.5 When SIMT Is Most Effective

The SIMT approach is most effective for highly data-parallel workloads with regular, predictable memory access patterns and limited control-flow divergence within warps. Under these conditions, the hardware can keep most lanes active, achieve high memory bandwidth utilization through coalesced accesses, and deliver both high performance and a relatively simple thread-based programming model, which is why SIMT has become the dominant architecture for GPU-based applications.

2.3 SCHEDULE STAGE

The scheduler operates in a cyclic manner to select eligible warps for dispatch. It first determines the set of ready warps as those that are active and not currently stalled, then uses a priority encoder to implement round-robin selection among this ready set. The selected warp is immediately marked as stalled to enforce single-cycle dispatch and prevent it from being reissued before the pipeline acknowledges the request. Once its instruction has been accepted and processed, the scheduler receives an acknowledgment signal, clears the stall condition for that warp, and returns it to the ready pool. This mechanism provides fair access to execution resources while avoiding instruction duplication.

2.3.1 Program Counter (PC)

In the Vortex GPGPU architecture, the Program Counter (PC) is managed on a per-warp basis and is primarily used within the Schedule (VX_schedule) and Fetch (VX_fetch) stages of the 6-stage SIMT pipeline. The scheduler maintains an independent PC for each warp and, on every cycle, selects a ready warp and forwards its PC and thread mask to the Fetch stage, allowing different warps to progress through the program at different points in time. The Fetch stage then uses this PC to access the instruction cache, handles hits and misses, and feeds instructions into the pipeline. Because Vortex implements the RISC-V ISA (RV32/RV64), the PC normally advances by 4 bytes for standard instructions, or by 2 bytes when the compressed (C) extension is enabled, while branches and jumps are resolved in the Execute stage, which updates the PC either to a target address or to the next sequential instruction. Under SIMT execution, divergence within a warp is handled by tracking both the PC and the associated thread mask for different control paths and then reconverging threads at the correct point in the instruction stream, preserving the illusion of independent scalar threads on top of warp-based hardware.

2.3.2 Barrier Synchronization

In the Vortex GPGPU architecture, **barrier synchronization** is a core functional component of the pipeline, primarily managed and enforced within the **Schedule (VX_schedule)** stage.

The following details from the sources describe how barrier synchronization is integrated into the system:

Management in the Schedule Stage

The **Schedule stage** is responsible for orchestrating the execution of multiple warps. It specifically manages **barriers** alongside warp scheduling and **thread mask convergence (TMC)**. Its primary role in synchronization is to determine **warp availability**; it controls which warp executes next based on whether it has reached a synchronization point and is waiting for other warps to arrive.

Synchronization and the SIMT Model

Because Vortex uses a **SIMT (Single Instruction, Multiple Threads)** model, synchronization is essential for maintaining order among parallel workloads.

- **Warp Level:** The scheduler tracks the status of each warp (configured via NUM_WARPS) to ensure that warps hitting a barrier are paused until the synchronization condition is met.
- **Thread Level:** Within each warp, the TMC mechanism ensures that diverged threads eventually reconverge, which is often a prerequisite for or a result of barrier operations.

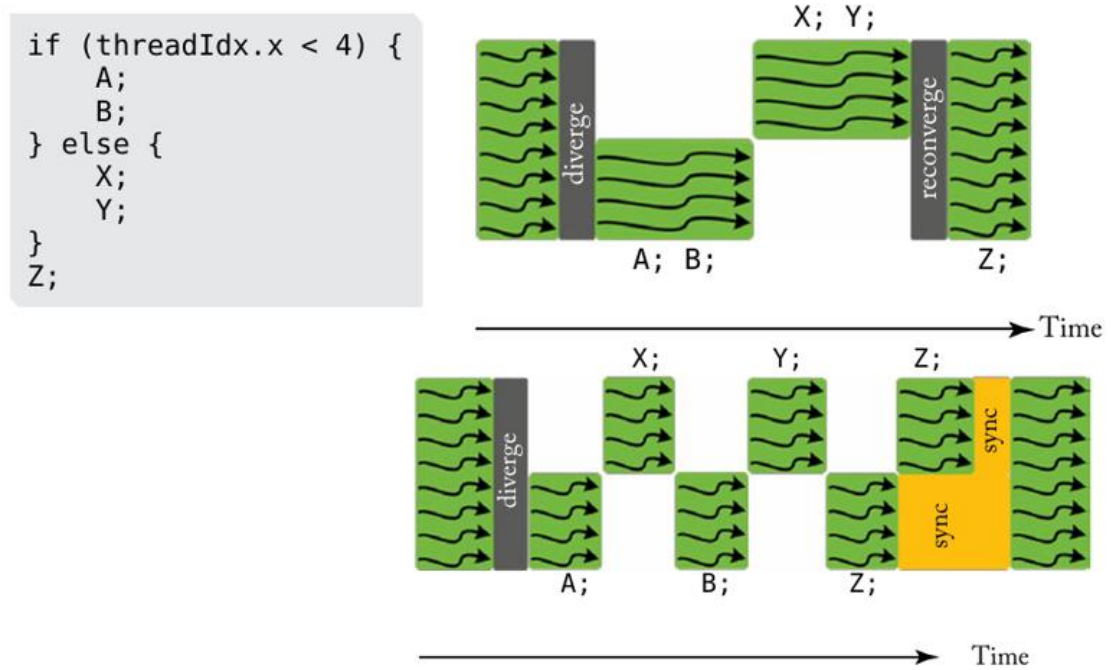


Figure 7, Barrier Synchronization.

In this example, **barrier synchronization** enforces correct execution order after divergence by ensuring that **all threads in the warp reach the same synchronization point before continuing**. After the conditional branch diverges, the Vortex GPU serializes the execution of both paths (A–B and X–Y) with thread masking. Even though different threads execute different instructions, the hardware maintains per-thread execution state and program counters until all active paths complete. The **barrier (sync)** point prevents any thread from executing instruction **Z** until **both branches have fully finished**, guaranteeing reconvergence and memory consistency across the warp. In Vortex, this is typically implemented using **warp-level masks, reconvergence control logic, and synchronization counters**, ensuring deterministic execution and correctness in the SIMT model.

2.4 FETCH STAGE

The Fetch stage is the second stage in the pipeline, acting as the stateful bridge between the Warp Scheduler and the Instruction Cache (I-Cache). Its primary objective is to retrieve 32-bit instructions for scheduled warps while maintaining the metadata required for the subsequent stages.

2.4.1 Role and Context

The Fetch stage is the first active stage in the execution pipeline after warp scheduling. Its primary responsibility is to retrieve instructions from the Instruction Cache (I-Cache) for the warp selected by the scheduler. Unlike later stages, the Fetch stage does not interpret or decode instructions; instead, it acts as a pure memory interface, responsible only for moving instruction data reliably and efficiently.

A defining characteristic of the Fetch stage is that it must manage multiple outstanding instruction fetch requests from different warps while maintaining correctness even when responses return out of order. As such, the Fetch stage behaves as a stateful bridge between the scheduler and the memory subsystem.

2.4.2 Architecture & Data Flow

Schedule Interface (Input):

UUID: Unique instruction identifier for tracking

WID: Warp ID (which warp is being scheduled)

TMASK: Thread mask indicating which threads are active

PC: Program counter (address of the instruction to fetch)

Fetch Interface (Output):

Uuid, wid, tmask and PC: (passed through)

instr: instruction word fetched from memory

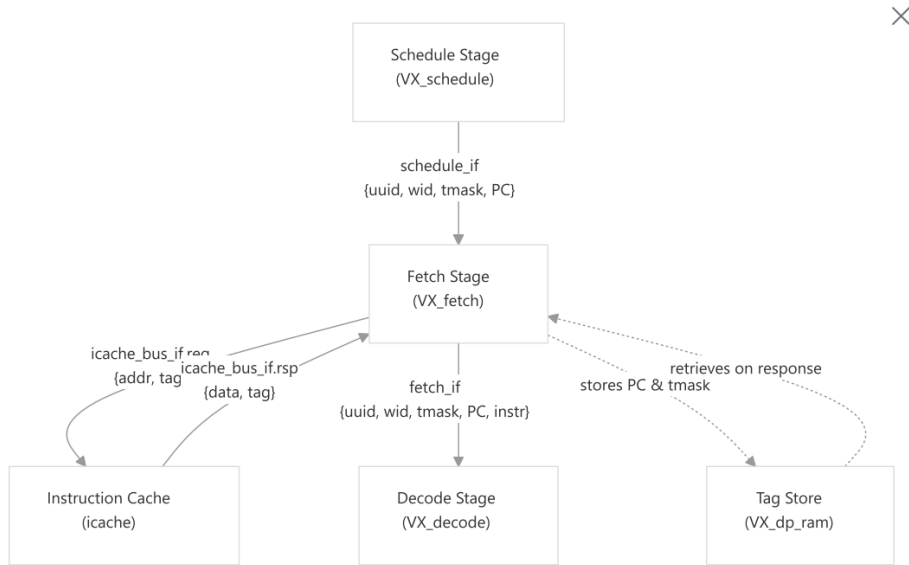


Figure 8, Fetch stage architecture

2.4.3 Fetch operation

Request Generation:

When a valid warp is scheduled, the fetch stage generates an icache request

The request includes:

Address: Extracted from the PC (4-byte aligned)

Tag: Combines UUID and warp ID for tracking the request

Tag Storage:

The fetch stage stores the PC and thread mask in a dual-port RAM indexed by warp ID.

This is necessary because icache responses may arrive out-of-order

The tag store is written on every icache request and read when responses arrive, allowing the fetch stage to reconstruct the full fetch information.

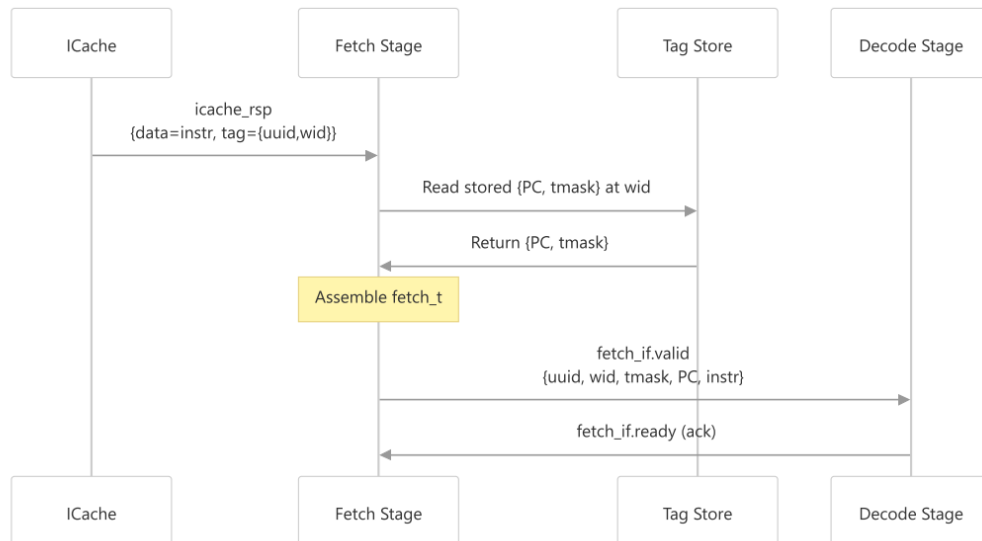
Response Processing:

Extracts the UUID and warp ID from the response tag

Retrieves the stored PC and thread mask from the tag store

Assembles the complete `fetch_t` structure

Outputs it to the decode stage



2.4.4 Request Buffering

Requests are buffered through an elastic buffer before being sent to the icache bus.

- This decouples the fetch stage from icache timing and helps maintain throughput.
- Prevent Pipeline Deadlock Especially when: L1 cache is disabled or

I-cache and D-cache share the same bus

- IBUF flow control: Limits outstanding fetches per warp, Prevents buffer overflow and
- Avoid circular stalls.

2.4.5 Complete Data Flow

- When L1 cache is disabled, the fetch stage implements flow control to prevent the instruction buffer (ibuffer) from overflowing.
- This mechanism tracks pending fetches per warp and stalls scheduling when a warp's ibuffer is full, preventing deadlock when icache and dcache share the same bus.

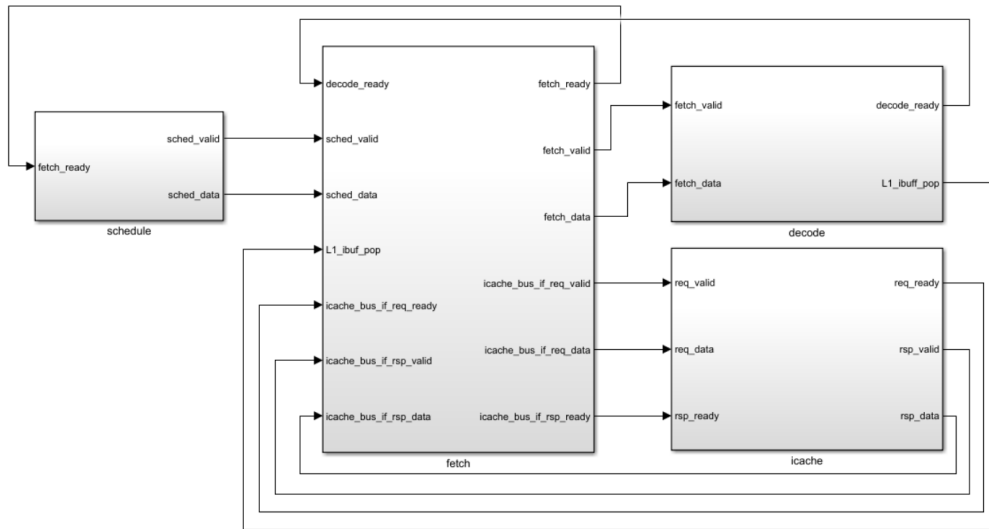


Figure 9, Complete fetch data flow diagram

2.5 DECODE STAGE

The Decode stage is the "translator" of the pipeline. It is a purely combinational stage that breaks down a raw of binary bits instruction into control signals and immediate values.

2.5.1 Role & context

Primary Responsibility:

- Translates raw instructions into execution-ready control information
- Determines: Execution unit, Operation type and Register operands
- Communicates control-flow information back to scheduler

Key Characteristic: It is purely combinational logic. It translates the instruction in a single clock cycle, with no internal state between instructions.

2.5.2 The Decoding Process

Input Interface (fetch_if): uuid, wid, tmask, PC and instr.

Step 1: Extract Instruction Fields

The decoder extracts standard RISC-V fields:

- opcode (bits 6:0): Primary operation code
- Funct_x: each x Function code for operation variants
- rd, rs1, rs2, rs3: Register indices

Step 2: Decode Immediate Values

Based on instruction type, different immediate formats are extracted:

- I-type: 12-bit immediate (with special handling for shift instructions)
- S-type: Store immediate
- B-type: Branch immediate
- U-type: Upper immediate
- J-type: Jump immediate

Step 3: Determine Execution Unit and Operation Type

The decoder performs a large case statement on the opcode to determine Execution unit

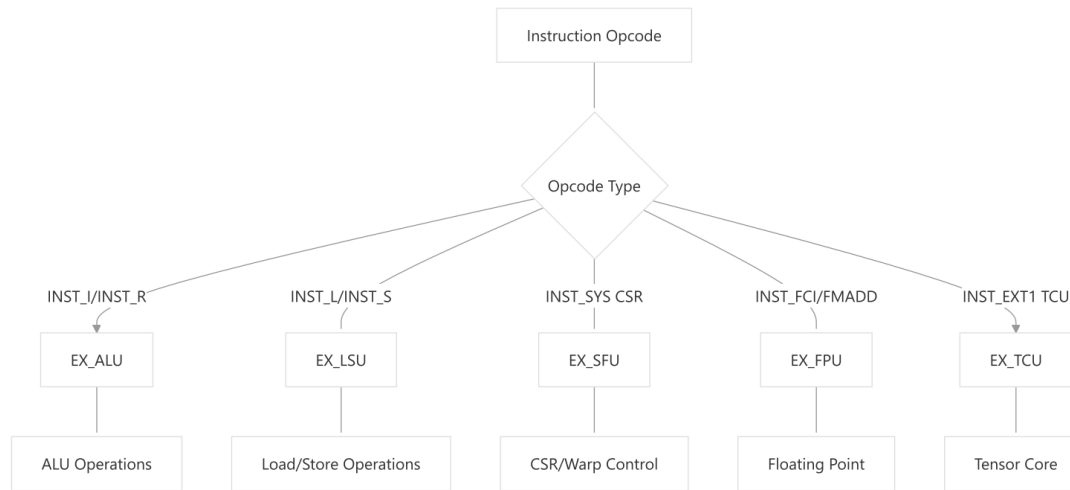


Figure 10,execution unit selection

Step 4: Identify Register Usage

For each instruction type, the decoder identifies which registers are used.

The `USED_IREG` and `USED_FREG` macros mark registers as integer or floating-point registers.

Step 5: Extract Operation-Specific Arguments

The decoder populates the `op_args` union based on execution unit

Step 6: Detect Warp-Stalling Instructions

Certain control flow instructions cause warp stalls

Instructions like `JAL`, `JALR`, branches, and some `SFU` operations set the `is_wstall` flag or unlock the warp.

2.5.3 Key Responsibilities of decode stage

Generate Output Interface (decode_if)

Table 1, Decode stage outputs

Field	Width	Description
uuid	UUID_WIDTH	A unique identifier for the instruction, used for tracking and debugging throughout the pipeline.
wid	NW_WIDTH	Warp ID: Identifies which warp the instruction belongs to.
tmask	NUM_THREADS	Thread Mask: A bitmask where each bit indicates if a specific thread in the warp is active for this instruction.
PC	PC_BITS	Program Counter: The memory address of the instruction.
ex_type	EX_BITS	Execution Unit Type: Selects the target unit (ALU, LSU, FPU, SFU, or TCU).
op_type	INST_OP_BITS	Operation Type: The specific operation code for the selected execution unit (e.g., ADD for ALU, LW for LSU).
op_args	Variable	Operation Arguments: A union containing unit-specific data (e.g., immediates, memory offsets, or branch flags).
wb	1 bit	Writeback Enable: A flag that tells the Commit stage whether to write the result back to the register file (cleared for \$r0\$ or branch instructions).
used_rs	3 bits	A bitmask indicating which source registers are active (rs1, rs2, and/or rs3).
rd	REG_ADDR_BITS	The index of the Destination Register .
rs1	REG_ADDR_BITS	The index of Source Register 1 .
rs2	REG_ADDR_BITS	The index of Source Register 2 .

rs3	REG_ADDR_BITS	The index of Source Register 3 (used primarily for Fused Multiply-Add instructions).
------------	---------------	---

Communication with Scheduler:

decode_sched_if Signals:

wid: Warp ID

valid: Whether warp can continue

unlock: Whether it must stall

Scheduler Uses This To:

- Manage warp state
- Avoid incorrect instruction issue

Preserve correct SIMT execution.

2.6 ISSUE STAGE

The issue stage is the 4th stage in the Vortex RISC-V pipeline. It plays the role of instruction flow management, keeping the whole pipeline alive through the process. It also reassures that the writeback operation is done smoothly. This stage implements the critical data path between decode and execution stages. It consists of three main blocks: Instruction Buffer, Scoreboard and Operands Collector.

2.6.1 Instruction Buffer

The instruction buffer (I-Buffer) is not only a queue buffer used for keeping instructions, but also it can keep and handle micro-operations between instructions using micro-operation sequencer. The I-Buffer stores decoded instructions in separate per-warp queues, decoupling the decode stage from the issue stage.

2.6.2 Scoreboard

Scoreboard is the conflict reliever in the process where instructions are dependent on each other as it checks for data dependencies and registers availability. If the register is Free and there are no data dependencies, then dispatch it otherwise; stalling occurs. Bitmapping is used to mark all the busy registers for smooth writeback operation preventing RAW and WAR hazards.

2.6.3 Operands Collector (OPC)

Operands Collector does the heavy work or the real collection of operands to be dispatched into the execution stage. Each OPC entry holds metadata and buffers for one instruction. GPUs use banked register files to provide high bandwidth operations. Operands are mapped into several banks to avoid requesting the same bank or any conflicts. Operands collection is done through 3 phases: Request Generation where OPC generates read requests for registers and mapped into several banks, Bank Arbitration where each mapped bank arbitrates to resolve conflicts and Data Return where the read request comes back and processed to collect and fetch data from banks.

2.6.4 Dataflow Overview

Decode stage → I-Buffer: Instructions get stored in per-warp queues.

I-Buffer → Scoreboard: Instructions flow from per-warp queues to dependency checking.

Scoreboard → OPCs: Only instructions with available source registers proceed.

OPCs → Dispatch: Instructions with fetched operands are routed to execution units.

Writeback → Scoreboard/OPCs: Completion signals update dependency tracking and register files.

2.7 EXECUTE STAGE

It is where actual computation takes place in the Vortex GPGPU pipeline.

It receives *ready-to-execute* instructions from the **Issue Stage**, routes them to the appropriate **functional units (FUs)**, and manages their execution based on operation type and latency.

Each instruction is executed in parallel across **multiple specialized units**, allowing Vortex to sustain high throughput and efficiently utilize its SIMD datapath.

The **Execute Stage** is structured around a **dispatch layer** that connects input interfaces (`dispatch_if`) from the Issue Stage to multiple **functional units (FUs)**.

Each FU performs a specific category of operations. Once executed, results are sent out via **commit interfaces (`commit_if`)** to the next stage (Commit).

2.7.1 Core Responsibilities

- Receive instructions from all issue slots (up to `ISSUE_WIDTH`)
 - Decode the **execution type (`ex_type`)** field to determine the target functional unit
 - Route the instruction and operands accordingly
 - Manage multi-cycle execution and packet iteration for SIMD threads
- This modular design enables **parallel execution** of multiple instruction types across different units simultaneously.

2.7.2 Functional Units

The Execute Stage contains **five major functional units**, each specialized for a distinct class of operations:

1. ALU Unit (Arithmetic Logic Unit)

Handles **integer arithmetic, logic operations, and branch control**.

Operations:

- Integer arithmetic: add, sub, shift, bitwise logic (AND, OR, XOR)
- Multiply/divide operations (configurable latency)
- Branch evaluation and PC update (communicates with scheduler)

Latency:

- Arithmetic & logic: ~2 cycles
- Multiply: 2 cycles (configurable)
- Divide: $XLEN + 2$ cycles (≈ 34 cycles for 32-bit)

Key Role:

Implements core integer datapath and branch resolution logic for each warp.

2. LSU Unit (Load/Store Unit)

Handles **memory-related operations**:

- Loads, stores, and **atomic memory** operations
- Interfaces with the **data cache** and **memory hierarchy** through `lsu_mem_if`

Operation Flow:

1. Receives memory instructions from dispatch
2. Performs address calculation
3. Sends/receives data via cache interface
4. Forwards data/results to Commit stage

Latency:

Variable — depends on memory hierarchy response (cache hits, misses, or atomic contention).

3. FPU Unit (Floating-Point Unit)

Handles **IEEE 754 floating-point** operations (if FPU extension is enabled).

Operations:

- Floating-point add, subtract, multiply, divide
- Square root and conversion operations (e.g., float ↔ int)

Latency:

4–10 cycles depending on operation complexity.

Notes:

FPU execution occurs in parallel with integer ALU pipelines, maximizing utilization for mixed workloads.

4. SFU Unit (Special Function Unit)

Handles **system-level and warp-control** operations such as:

- **CSR** (Control and Status Register) read/write
- **Warp control instructions:** TMC, WSPAWN, SPLIT, JOIN, BARRIER
- **Performance counter updates**

Latency:

2–4 cycles depending on instruction complexity.

Key Role:

Coordinates control flow and warp management, making it a bridge between the scheduler and the hardware control plane.

5. TCU Unit (Tensor Compute Unit)

Handles **tensor operations** for AI and matrix-heavy workloads (when tensor extension is enabled).

Operations:

- **Matrix multiply-accumulate (MAC)** operations across SIMD lanes
- Accelerates tensor-based arithmetic, crucial for ML inference workloads

Latency:

Multi-cycle, depends on tensor size and SIMD configuration.

2.7.3 Dispatch Mechanism

The **dispatch layer** is responsible for sending each instruction to its appropriate functional unit based on the **execution type (ex_type)** determined during decode.

Process:

1. Receive instructions from all **issue slots** (up to `ISSUE_WIDTH`)
2. Decode `ex_type` to identify the correct FU (ALU, LSU, FPU, SFU, TCU)
3. Route instruction and operand data to that FU's input queue
4. For wide warps (`NUM_THREADS > SIMD_WIDTH`), manage **SIMD iteration**:
 - Issue instructions across multiple cycles
 - Use **start-of-packet (SOP)** and **end-of-packet (EOP)** signals to track progress

This mechanism ensures efficient utilization of execution units and proper synchronization between SIMD lanes.

2.7.4 Execution Flow**In hardware:**

- Each **functional unit** works independently, processing its queue of incoming instructions.
- Execution is pipelined — multiple instructions can be in-flight simultaneously across different units.

In SIMX Simulator

The simulator mimics the hardware dispatch logic:

1. Checks if a dispatcher has output ready
 2. Pushes the instruction to the appropriate FU input queue
 3. Each FU processes over several cycles (based on latency)
 4. Results are pushed to the Commit stage for writeback and synchronization
- This allows accurate timing and performance modeling.

2.7.1 Latency Characteristics

Table 2, Latency Characteristics

Functional Unit	Typical Latency (cycles)	Notes
ALU	2	Add, logic, shifts
Multiply	2 (configurable)	Integer multiplication
Divide	$XLEN + 2$ (~34 for 32-bit)	Iterative division
FPU	4–10	FP operations (variable)

LSU	Variable	Depends on cache/memory latency
SFU	2–4	CSR/warp control ops
TCU	Multi-cycle	Tensor MAC

2.7.2 Parallel Execution and Warp Distribution

Each functional unit can handle instructions from **multiple warps simultaneously**, enabling **fine-grained multithreading**.

Different instruction types execute in parallel, e.g.:

- One warp may perform a **memory load** (LSU)
- Another executes a **floating-point operation** (FPU)
- A third may issue a **branch** (ALU)

This design achieves high throughput and hides long-latency operations (e.g., memory or division) using **warp-level parallelism**.

2.7.3 Key Takeaways

- The **Execute Stage** is the *core computational engine* of the Vortex pipeline.
- It interfaces:
 - **Upstream** with the **Issue Stage** via `dispatch_if`
 - **Downstream** with the **Commit Stage** via `commit_if`
 - **Externally** with memory and scheduler subsystems
- Its design ensures:
 - **Concurrent execution** of multiple instruction types
 - **Efficient SIMD management**
 - **Configurable latency handling** for complex operations

2.8 COMMIT STAGE

The Commit stage is the final destination where parallel results from various execution units converge to update the processor state.

2.8.1 Role & context

Primary Responsibilities:

- **Arbitration:** Selecting one result per issue slot from potentially competing execution units.

- Retirement: Counting retired thread-instructions for architectural performance counters.
 - Writeback: Triggering the update of the architectural register file.
 - Notification: Informing the scheduler which warps have completed instructions.
- Key Challenge: Handle out-of-order completion from different execution units while maintaining precise per-warp and per-thread accounting.

2.8.2 Inputs & Outputs

Inputs (commit_if [NUM_EX_UNITS * ISSUE_WIDTH]):

- A 2D Array of Interfaces: NUM_EX_UNITS (e.g., ALU, LSU) × ISSUE_WIDTH (parallel lanes).
- The Payload (commit_t): Contains the result data, destination register (rd), thread mask (tmask), and critical control flags (wb, eop).

Outputs (Three Critical Paths):

- writeback_if: Carries the arbitrated result to the Register File. Only produced if wb=1.
- commit_csr_if: Sends the retired instruction count (instret) to the CSR unit.
- commit_sched_if: Signals the Scheduler which warps (wid) have finished an instruction (when eop=1).

2.8.3 Arbitration

In the commit stage, multiple execution units such as the ALU, LSU, and FPU may complete execution for the same issue lane in the same clock cycle, creating contention for commitment. To resolve this, the architecture employs a priority arbiter per issue slot, implemented using the VX_stream_arb module. This arbiter examines all incoming results associated with a given issue lane and selects exactly one instruction to commit in that cycle. A static priority policy is used, where execution units are ordered deterministically (for example, ALU has higher priority than LSU, which has higher priority than FPU). This approach ensures predictable and repeatable behavior while avoiding complex arbitration logic. As a result, each issue slot produces at most one clean and well-defined commit_t packet per cycle, preserving pipeline correctness.

2.8.4 Instruction Retirement Counting

The commit stage is responsible for updating the RISC-V minstret CSR, which tracks the number of retired instructions. This process is performed in multiple steps to accurately reflect the SIMT execution model. First, for each issue slot, the number of active threads participating in the committed instruction is calculated by applying a population count (popcount) to the instruction's thread mask (tmask). Next, the per-slot counts are summed across all `ISSUE_WIDTH` lanes using a reduction tree to obtain the total number of retired thread-instructions for that cycle. This total is then accumulated into the instret register, updating the architectural retirement counter. To avoid extending the critical timing path, the entire counting and accumulation process is implemented using a two-cycle pipeline. Importantly, the counter tracks thread-instructions rather than warp-instructions, reflecting the true amount of work completed by the GPU.

2.8.5 Warp Tracking

In addition to counting retired instructions, the commit stage tracks which warps have completed an instruction. This is achieved by constructing a bitmask called `committed_warps`, where a bit corresponding to a warp ID is set when two conditions are met: the instruction successfully commits (`commit_fire`) and it represents the end of a multi-cycle instruction packet (`eop = 1`). This bitmask is forwarded to the scheduler through the commit scheduling interface. The scheduler interprets this signal as a receipt indicating that a warp has completed an instruction, allowing it to decrement the warp's pending instruction counter. This mechanism is essential for freeing warps and enabling them to be scheduled again, ensuring forward progress and preventing pipeline stalls.

2.8.6 Writeback

The final responsibility of the commit stage is register writeback generation. For each arbitrated commit result, if the writeback enable flag (`wb`) is asserted, the commit stage drives the `writeback_if` interface. This interface carries the execution result data along with the destination register (`rd`), warp identifier (`wid`), and thread mask (`tmask`) back to the Operands or Register File stage, where the architectural state is updated. A critical design choice in this stage is that `commit_arb_if.ready` is permanently asserted, meaning the commit stage never stalls execution units. This greatly simplifies pipeline control and avoids backpressure at commit time;

however, it requires execution units to include sufficient internal buffering to safely absorb downstream latency.

2.9 MEMORY SYSTEM

2.9.1 Introduction

The memory system in the RISC-V Vortex GPGPU is a critical component because it directly determines how efficiently large numbers of parallel threads can access data. GPGPU workloads typically involve massive data sets and thousands of concurrent memory requests generated by multiple warps executing in parallel. Without a well-designed memory hierarchy, these requests would quickly become a performance bottleneck. The Vortex memory system is therefore structured to provide high bandwidth and low latency, enabling compute cores to remain busy and minimizing stalls caused by memory access delays.

Another important aspect of the Vortex memory system is its hierarchical organization, which balances performance and scalability. By separating memory into different levels—such as local memory, shared or scratchpad memory, caches, and global memory—the architecture can exploit data locality at multiple levels. Frequently accessed data can be kept closer to the cores, reducing access time and energy consumption, while larger global memory supports data sharing across many threads and kernels. This hierarchy is especially important in GPGPU applications where the same data may be reused by multiple threads within a workgroup.

The memory system is also tightly coupled with the load/store unit (LSU) and the SIMT execution model used in Vortex. Since threads within a warp often issue memory requests simultaneously, the memory system must efficiently handle coalesced accesses while also supporting divergent or irregular access patterns. An optimized memory subsystem ensures that aligned and coalesced accesses are combined into fewer transactions, improving effective bandwidth, while still correctly servicing non-coalesced accesses when required.

2.9.2 Memory Hierarchy

The memory hierarchy shown illustrates how data is organized and accessed in a GPGPU system such as the RISC-V Vortex, moving from the fastest and smallest storage near the execution units to the largest and slowest memory at the system level. This hierarchical design is essential to balance performance, scalability, and energy efficiency when supporting massive parallel execution.

At the top of the hierarchy are the **registers**, which are private to each thread. Registers provide the lowest latency and highest bandwidth access, making them ideal for frequently used variables and intermediate computation results. Because each thread has its own register file, register access does not cause contention between threads, which is crucial for sustaining high throughput in SIMT execution.

Below the registers is the **local memory**, which is shared by all threads within a core. This memory is typically used for data that needs to be accessed by multiple threads running on the

same core. Compared to registers, local memory has higher latency but still offers fast access and predictable performance. It plays an important role in cooperative computations, such as sharing intermediate results among threads within a workgroup.

The next level is the **L1 cache**, shared by all cores within a socket. The L1 cache acts as a fast buffer for recently accessed data, reducing the number of accesses to lower, more expensive memory levels. By exploiting spatial and temporal locality, the L1 cache significantly improves effective memory bandwidth and reduces average memory access latency for many workloads.

Below L1 is the **L2 cache**, which is shared at the cluster level. The L2 cache provides a larger storage capacity than L1 and serves as an intermediary between the fast on-chip caches and global memory. It helps reduce traffic to global memory by capturing data shared across multiple cores and sockets, improving overall system scalability.

At the bottom of the hierarchy is the **L3 or global memory**, which is system-wide and accessible by all clusters. This memory offers the largest capacity but also the highest latency. Global memory is used to store large data sets and to enable communication between different kernels or execution phases. Efficient use of caches and higher memory levels is critical to

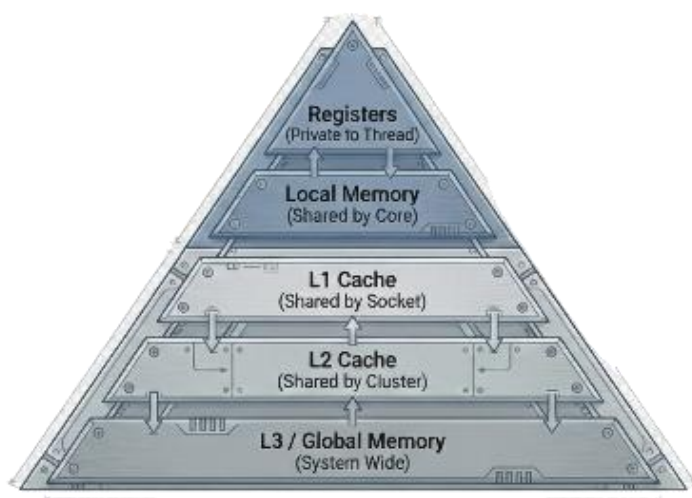


Figure 11, Memory Hierarchy

minimize accesses to global memory and avoid performance bottlenecks

2.9.3 L1 Cache

The L1 Data Cache (L1 D-Cache) is the closest cache level to the processor cores and provides the fastest automatic data storage mechanism in the memory hierarchy. It stores frequently accessed data to minimize memory access latency.

Key Characteristics

- **Capacity:** 32–64 KB per core
- **Access Latency:** 3–10 clock cycles
- **Bandwidth:** >100 GB/s per core
- **Sharing Model:** Private to each core
- **Management:** Fully managed by hardware (transparent to the programmer)

Operation:

When a thread issues a data request, the hardware checks the L1 D-Cache first: - **Cache Hit:** Data is returned within 3–10 cycles - **Cache Miss:** The request is forwarded to the L2 cache

Practical Example: Sequential Array Access

Sequential memory access benefits from spatial locality. When an element is fetched, the entire cache line containing neighboring elements is loaded into the L1 cache, significantly accelerating subsequent accesses.

Performance Impact

- Reduces memory access latency from hundreds of cycles (DRAM) to single-digit cycles
- Applications with good locality often achieve L1 hit rates above 90%

The L1 Instruction Cache stores program instructions rather than data. It feeds the instruction fetch unit with minimal latency, ensuring efficient pipeline execution.

Key Characteristics

- **Capacity:** 32–64 KB per core
- **Access Latency:** 3–10 clock cycles
- **Sharing Model:** Private to each core
- **Access Type:** Read-only

Table 3 ,Comparison with L1 D-Cache

Aspect	L1 I-Cache	L1 D-Cache
Stored Content	Instructions	Data values
Access Type	Read-only	Read/Write
Used By	Instruction fetch unit	Load/Store units

Importance

Efficient instruction caching prevents instruction fetch from becoming a bottleneck. Tight loops in GPU kernels typically achieve near-perfect instruction cache hit rates (>99%).

L2 Cache

The L2 Cache is a second-level cache shared among multiple cores. It serves as a larger and slower backup to L1 caches and enables data sharing between cores.

Key Characteristics

- **Capacity:** 512 KB–6 MB
- **Access Latency:** 10–50 clock cycles
- **Bandwidth:** 50–200 GB/s
- **Sharing Model:** Shared across multiple cores
- **Type:** Unified (stores both instructions and data)
- **Role in Memory Access Flow**

The L2 cache filters a significant portion of L1 cache misses, reducing the number of expensive DRAM accesses.

Performance Benefits

- Bridges the latency gap between L1 caches and DRAM
- Prevents DRAM bandwidth saturation by filtering 70–90% of L1 misses

L3 Cache

The L3 Cache, when present, is the last on-chip cache level shared across the entire GPU. It provides a large shared memory pool before accessing DRAM.

Key Characteristics

- **Capacity:** 4–64 MB
- **Access Latency:** 50–100 clock cycles
- **Bandwidth:** 20–100 GB/s
- **Sharing Model:** GPU-global

Use Cases

L3 cache is particularly effective for large working sets, multi-kernel workloads, and irregular access patterns that exceed L2 capacity.

DRAM

DRAM is the main external memory of the GPU, providing large storage capacity at the cost of high access latency. All data ultimately resides in DRAM.

Key Characteristics

- **Capacity:** 8–80 GB
- **Access Latency:** 200–800 clock cycles
- **Bandwidth:** 128 GB/s–4 TB/s
- **Technologies:** GDDR6, HBM2/HBM3

Table 4, DRAM Technology Comparison

Technology	Bandwidth	Capacity	Typical Use Case
GDDR6	Up to 1 TB/s	8–24 GB	Consumer GPUs
HBM2/HBM3	Up to 4 TB/s	16–80 GB	PC and AI workloads

Optimization Strategies

- Maximize cache hit rates
- Use coalesced memory accesses
- Reuse data to amortize DRAM cost
- Overlap computation with memory transfers

Shared Memory

Shared memory is a programmer-managed, on-chip scratchpad that enables fast and deterministic data sharing among threads within a workgroup.

Key Characteristics

- **Capacity:** 16–128 KB per compute unit
- **Access Latency:** 1–2 clock cycles
- **Bandwidth:** >500 GB/s
- **Sharing Model:** Per workgroup
- **Management:** Explicitly controlled by software

Table 5, Comparison with Caches

Aspect	Shared Memory	Cache
Management	Software	Hardware
Latency	Deterministic	Variable
Control	Explicit	Implicit

Typical Applications

Shared memory is ideal for reductions, tiled algorithms, inter-thread communication, and data reuse within a block.

2.9.4 Cache Bank Architecture

The Memory Bottleneck Challenge

Modern General-Purpose Graphics Processing Units (GPGPUs) face a fundamental architectural challenge that threatens to limit the benefits of massive parallelism: **memory access bottlenecks**. While GPGPUs are designed to execute thousands of threads concurrently across many processing cores, the effectiveness of this parallel execution is highly dependent on the memory system's ability to supply data at a matching rate. When memory access cannot scale with computational parallelism, overall system performance degrades significantly.

As the number of cores in a GPGPU increases, the aggregate demand for memory accesses grows proportionally. However, in a conventional cache architecture with a single access port, memory requests are inherently serialized. Even if each cache access requires only a single cycle, concurrent requests from multiple cores must be serviced sequentially. This serialization negates the advantages of parallel computation and leads to substantial underutilization of available compute resources.

To illustrate this limitation, consider a GPGPU configuration with four processing cores, each issuing a cache access in the same cycle. In a single-ported cache, only one request can be serviced per cycle. Consequently, the first core completes its access immediately, while the remaining three cores are forced to wait. The second core completes in the second cycle, the third in the third cycle, and the fourth in the fourth cycle. What should ideally be a one-cycle parallel operation is transformed into a four-cycle serialized sequence, resulting in 75% of the cores being idle at any given time. As the number of cores increases, this inefficiency grows linearly, severely limiting scalability.

This behavior directly contradicts the core design objective of GPGPUs, which is to exploit parallelism at scale. A memory system that can only service a single request per cycle cannot support the demands of dozens or hundreds of concurrently executing cores. In such a system, the cache intended to reduce memory latency and improve performance becomes the dominant performance bottleneck. As GPGPU architectures scale to 16, 32, or more cores, the mismatch between computational throughput and memory access capability becomes increasingly severe.

Addressing this challenge requires a fundamental rethinking of cache architecture. Rather than treating the cache as a single monolithic resource, it must be redesigned to support multiple simultaneous accesses. Techniques such as cache banking, multi-porting, and non-blocking interconnects are essential to overcoming this bottleneck and enabling memory systems that scale with parallel computation.

The Multi-Bank Cache Solution

A key architectural innovation that enables parallel memory access in GPGPUs is the use of **multi-bank cache architectures**. Rather than implementing a single monolithic cache that services all memory requests sequentially, the cache is partitioned into multiple independent banks. Each bank operates as a fully functional cache, allowing multiple memory requests to be processed concurrently. This transformation converts cache access from a fundamentally serial operation into a parallel one, directly addressing the memory bottleneck inherent in highly parallel processors.

In a four-bank cache design, the cache is not merely divided into four smaller storage regions; instead, it is implemented as four independent cache instances. Each bank can simultaneously service a request from a different core. For example, while Bank 0 performs a tag lookup for one core, Bank 1 may be servicing a data read for another core, Bank 2 may be resolving a cache miss, and Bank 3 may be updating replacement state. This concurrent operation across banks enables the cache to sustain multiple accesses per cycle, thereby scaling throughput with the number of banks.

Each cache bank replicates the full internal structure of a conventional cache. Specifically, every bank includes its own tag array for cache line identification, a data array for storing cached data, Miss Status Holding Registers (MSHRs) to track outstanding misses, and replacement policy logic to manage cache evictions. Because these components are fully replicated, banks can progress through their cache access pipelines independently. This independence ensures that different pipeline stages such as tag comparison, data access, and miss handling can be executed concurrently across banks within the same cycle.

True parallelism requires strict independence among banks. Any shared structure, such as a common tag comparison unit, a shared data path, or centralized miss-handling logic, would reintroduce serialization and limit throughput. To achieve the full performance benefits of banking, each bank must be capable of completing an entire cache access without relying on shared resources. Only under this condition does banking provide a near-linear increase in throughput with the number of banks.

Address Mapping and Bank Selection

While independent banks enable parallelism, effective performance also depends critically on the **address-to-bank mapping strategy**. The mapping function determines which bank services a given memory address and directly influences load balance, conflict rates, and overall cache efficiency. An effective mapping must distribute accesses evenly across banks while preserving spatial locality, which is essential for cache effectiveness.

In the Vortex architecture, bank selection is performed using the **middle bits of the memory address**. For a four-bank configuration with a 64-byte cache line, address bits [5:4] are used to select the target bank. This choice avoids two pathological extremes. Using the lowest address bits would cause bytes within the same cache line to map to different banks, providing no practical benefit because cache lines are fetched as a unit. Conversely, using the highest address bits would map large contiguous memory regions to the same bank, leading to severe bank imbalance and increased contention.

Selecting the middle address bits achieves an optimal balance. With this scheme, consecutive cache lines are distributed cyclically across banks. As a result, common access patterns such as

sequential array traversal, matrix processing, or streaming workloads naturally generate parallel accesses across multiple banks. This behavior allows software to benefit from cache parallelism without requiring explicit programmer or compiler intervention.

To illustrate this mechanism, consider a sequence of cache line addresses. Address 0x1000 maps to Bank 0, 0x1040 maps to Bank 1, 0x1080 maps to Bank 2, and 0x10C0 maps to Bank 3. The next cache line at 0x1100 then maps back to Bank 0, and the pattern repeats. This cyclic distribution ensures even bank utilization for sequential memory accesses, effectively transforming serial software access patterns into parallel hardware execution.

Achieving Perfect Parallel Access

When a banked cache architecture and its address-mapping strategy operate in harmony, the system can achieve **perfect parallel access**. This ideal condition occurs when simultaneous memory requests from multiple cores are distributed evenly across different cache banks, allowing all requests to be serviced concurrently. Examining this case illustrates both the performance potential of cache banking and the mechanisms required to sustain high throughput.

Consider a four-bank cache serving four processing cores, each issuing a load request in the same cycle. Let Core 0 request address 0x1000, Core 1 request 0x2040, Core 2 request 0x3080, and Core 3 request 0x40C0. Using a bank-selection function based on address bits [5:4], these addresses map to Banks 0, 1, 2, and 3, respectively. This mapping results in an even distribution of requests with no bank conflicts.

In the following cycle, all four banks process their requests concurrently. Each bank independently extracts the index bits from its assigned address, performs a tag lookup in its local tag array, and determines whether the access results in a cache hit or miss. These tag comparisons occur in parallel across all banks, with no inter-bank dependencies.

Assuming cache hits in all four banks, each bank then accesses its data array and prepares the corresponding response. Because the banks operate independently, all four cores receive their requested data within the same cycle. In contrast to a single-ported cache, where four simultaneous requests would require four serialized cycles, the banked cache completes the operation in a single cycle. This corresponds to a **4× speedup** and demonstrates full utilization of available cache bandwidth.

In this ideal scenario, the cache achieves a throughput of four requests per cycle, which equals the theoretical maximum for a four-bank design. With 100% bank utilization and zero conflicts, the memory system transitions from being a performance bottleneck to an enabler of parallel execution. This behavior explains why banking is a fundamental design choice in high-performance GPGPU architectures.

Managing Bank Conflicts

While perfect parallelism represents the optimal case, real-world workloads often generate **bank conflicts**, which occur when multiple cores simultaneously issue requests that map to the same cache bank. Understanding how these conflicts are resolved and how they affect performance is critical for both hardware architects and software developers.

As an example, consider a scenario in which Core 0 and Core 1 both issue requests that map to Bank 0, while Core 2 and Core 3 issue requests mapping to Banks 2 and 3, respectively. Since each bank contains a single tag array, data array, and set of access ports, Bank 0 cannot service both requests in the same cycle.

To resolve this contention, each bank includes **arbitration logic** that selects one request to service when multiple requests arrive concurrently. In the Vortex architecture, a priority-based arbitration scheme is employed, where requests from lower-indexed cores are granted precedence. In this case, Bank 0 services Core 0's request in the first cycle, while Core 1's request is placed in a local queue. Meanwhile, Banks 2 and 3 continue to service their requests without interruption.

In the subsequent cycle, Bank 0 processes Core 1's queued request. From Core 1's perspective, the cache access latency has increased from one cycle to two cycles due to the conflict. Importantly, this delay is localized: cores accessing other banks are unaffected. As a result, a bank conflict does not induce a global stall across the system.

From a throughput perspective, the system services four requests over two cycles, yielding an average throughput of two requests per cycle. Although this is lower than the ideal four requests per cycle, it remains significantly higher than the single request per cycle achievable with a single-ported cache. This behavior illustrates a key advantage of banking: **graceful degradation**. Even in the presence of conflicts, the architecture preserves partial parallelism rather than collapsing into fully serialized execution.

While perfect parallelism represents the optimal case, real-world workloads often generate bank conflicts, which occur when multiple cores simultaneously issue requests that map to the same cache bank. Understanding how these conflicts are resolved and how they affect performance is critical for both hardware architects and software developers.

Inside a Cache Bank

To fully understand how cache banking enables parallel memory access, it is necessary to examine the internal structure of an individual cache bank. A cache bank is not a simple storage array; rather, it is a deeply pipelined microarchitectural unit composed of multiple stages, each responsible for a critical function in the cache access process. The internal organization of a bank directly determines both its access latency and its ability to sustain high throughput under diverse memory access patterns.

At the input of each cache bank resides a **request queue** and an associated **arbiter**. When multiple memory requests arrive that map to the same bank, they are enqueued in this request buffer. The arbiter examines all pending requests and selects one for service according to a predefined arbitration policy, such as priority-based or round-robin selection. The chosen request is admitted into the bank's processing pipeline, while the remaining requests remain queued.

This queuing mechanism provides essential buffering, allowing the bank to absorb short-term bursts of requests and to maintain throughput in the presence of bank conflicts. Without such buffering, transient contention would immediately propagate stalls back to the cores.

Once admitted to the pipeline, the selected request proceeds to the **tag lookup stage**. Using the index bits of the address, the bank accesses a specific set within its tag array. The tag array is typically implemented using SRAM to minimize access latency. In a W -way set-associative cache, the tag lookup stage retrieves W tags simultaneously, one for each way in the indexed set.

These tags are forwarded to the **comparison stage**, where dedicated comparator circuits operate in parallel to compare each stored tag against the request's tag. Each comparator independently checks for equality and valid state, enabling all ways to be evaluated within a single cycle.

The outputs of the comparators are examined by the **hit/miss determination logic**. If at least one comparator reports a match, and the corresponding valid bit is set, the request is classified as a cache hit. The logic identifies the matching way and generates control signals for subsequent pipeline stages. If no match is detected, the request is classified as a cache miss, and the pipeline diverges into a miss-handling path.

This decision point represents a critical branch in the cache pipeline, separating the fast hit path from the more complex miss path.

On a cache hit, the bank proceeds to the **data array access stage**. Using the index and the identified way number, the bank accesses the corresponding location in the data SRAM and retrieves the full cache line. The requested word or byte is then extracted using the offset bits of the address and returned to the requesting core.

Concurrently, the bank updates its **replacement policy state** (e.g., LRU or pseudo-LRU), marking the accessed cache line as recently used. This update occurs in parallel with data delivery to avoid extending the critical path.

At the input of each cache bank resides a request queue and an associated arbiter. When multiple memory requests arrive that map to the same bank, they are enqueued in this request buffer. The arbiter examines all pending requests and selects one for service according to a predefined arbitration policy, such as priority-based or round-robin selection. The chosen request is admitted into the bank's processing pipeline, while the remaining requests remain queued.

This queuing mechanism provides essential buffering, allowing the bank to absorb short-term bursts of requests and to maintain throughput in the presence of bank conflicts. Without such buffering, transient contention would immediately propagate stalls back to the cores.

At the input of each cache bank resides a request queue and an associated arbiter. When multiple memory requests arrive that map to the same bank, they are enqueued in this request

buffer. The arbiter examines all pending requests and selects one for service according to a predefined arbitration policy, such as priority-based or round-robin selection. The chosen request is admitted into the bank's processing pipeline, while the remaining requests remain queued.

This queuing mechanism provides essential buffering, allowing the bank to absorb short-term bursts of requests and to maintain throughput in the presence of bank conflicts. Without such buffering, transient contention would immediately propagate stalls back to the cores.

B. Tag Lookup and Comparison

Banking Combined with MSHR

In the Vortex architecture, each cache bank maintains its own independent MSHR structure with multiple entries. A typical configuration includes four MSHR entries per bank. In a four-bank cache, this organization allows up to sixteen concurrent cache misses to be tracked simultaneously. This represents a substantial improvement over traditional blocking caches, which stall on a single miss.

The performance benefits of cache banking are significantly amplified when combined with Miss Status Holding Registers. While banking enables parallel access to different addresses, MSHRs allow the cache to tolerate memory latency by supporting multiple outstanding cache misses. Together, these mechanisms create a non-blocking, highly parallel cache architecture suitable for GPGPU workloads.

The Crossbar Interconnect

Banking divides a cache into multiple independent banks to increase parallel access; however, an interconnection mechanism is required to efficiently connect multiple requesting cores to these banks. The **crossbar interconnect** fulfills this role by providing a non-blocking switching fabric that enables simultaneous communication between cores and cache banks. As such, the crossbar represents a critical component of the overall banking architecture.

A crossbar is a switching network composed of N inputs and M outputs, where any input can be dynamically connected to any output. In a representative system consisting of 16 processing cores and 4 cache banks, a 16×4 crossbar is required. Each core issues memory requests to the crossbar, which inspects the bank-selection bits of the request address to determine the target bank. The crossbar then routes each request to its corresponding bank.

A key property of the crossbar interconnect is its ability to support **multiple concurrent transfers**, provided that the requests target different banks. For example, if four distinct cores issue requests to four different banks, all transfers can proceed simultaneously without interference. This capability preserves the parallelism introduced by cache banking and prevents the interconnecting from becoming a performance bottleneck.

When multiple cores issue requests to the same bank, **bank-level arbitration** is performed. In such cases, the arbitration logic associated with the contended bank selects one request for service, typically using a priority-based or round-robin arbitration scheme. The remaining

requests are temporarily buffered in queues until the bank becomes available. Importantly, this arbitration is localized to the contended bank, allowing requests to other banks to continue without delay. As a result, parallelism is maintained even in the presence of bank conflicts.

The non-blocking nature of the crossbar distinguishes it from traditional shared buses, where a single transfer monopolizes the interconnect and stalls all other traffic. In contrast, the crossbar enables independent, simultaneous transfers across different input–output pairs. Although the hardware complexity of a crossbar scales with the product of the number of cores and banks, this cost is justified by the ability to achieve full parallel throughput and fully exploit the benefits of cache banking.

Banking Throughout the Memory Hierarchy

The banking strategy applied to cache structures extends consistently throughout the entire Vortex memory hierarchy. This system-wide application of banking principles ensures that parallelism is preserved from the processor’s register file, through multiple cache levels, and down to external DRAM. Although the motivation for banking is similar at each level namely, increasing throughput and reducing contention the specific implementations vary according to the access patterns and sharing requirements of each level.

At the **register file level**, banking enables parallel operand access required for Single Instruction, Multiple Data (SIMD) execution. For example, in a register file consisting of 32 registers organized into 4 banks, registers are distributed cyclically across banks. Registers 0, 4, 8, 12, 16, 20, 24, and 28 reside in Bank 0; registers 1, 5, 9, 13, 17, 21, 25, and 29 reside in Bank 1; and so on. When multiple threads within a warp execute the same instruction and request operands from different banks, the hardware can service these accesses concurrently. For instance, an instruction that reads registers R1, R5, and R9—each mapped to a distinct bank—can be satisfied with three parallel reads. This parallel access capability is critical for sustaining high-throughput SIMD execution without pipeline stalls.

Local memory (shared memory or scratchpad) also employs banking to support concurrent accesses from multiple threads within a thread block. In a typical configuration with 64 KB of local memory divided into four banks, addresses are mapped to banks using low-order address bits. When threads cooperate on shared data structures, careful data layout ensures that simultaneous accesses are distributed across different banks. For example, in a tiled matrix multiplication kernel, tile elements can be arranged such that threads loading a tile row access distinct banks, thereby avoiding bank conflicts and enabling parallel memory accesses.

Higher levels of the memory hierarchy, including **L1, L2, and L3 caches**, also rely on banking to achieve sufficient throughput. The number of banks generally increases at higher cache levels to accommodate greater degrees of sharing and higher aggregate request rates. An L1 cache may employ two to four banks to serve a small number of cores, whereas an L2 cache may utilize four to eight banks to handle requests from multiple L1 caches. At the L3 level, which services the entire GPU, eight to sixteen banks may be required to prevent the cache from becoming a system-wide bottleneck. The bank count at each level is therefore carefully chosen to match the parallelism demands imposed by the levels below.

This hierarchical application of banking establishes a consistent architectural design philosophy across the memory system. At every level, resources are partitioned to enable parallel access,

address mapping is used to balance load across banks, and crossbar or network-based interconnects are employed to maintain non-blocking communication. As a result, the memory hierarchy scales effectively from small, low-latency structures near the processor to large, high-latency structures interfacing with DRAM, while preserving parallel throughput throughout. Without this comprehensive banking strategy, contention at any single level could introduce serialization that undermines the performance of the entire parallel architecture.

2.9.5 LSU Slice Architecture

Core Concept

The LSU (Load-Store Unit) slice is a key execution component in the Vortex RISC-V GPGPU architecture that is responsible for handling all memory access operations generated by parallel threads. In a GPGPU environment, many threads execute simultaneously and frequently issue load and store requests. The LSU slice enables these memory operations to be processed in parallel, ensuring that each thread can correctly read from or write to memory without interfering with others. It manages address generation, data sizing, alignment, and access type (load or store), allowing high-throughput memory access while preserving correctness.

The LSU slice is designed to work naturally with the SIMT (Single Instruction, Multiple Threads) execution model. When a group of threads issues a memory instruction, the LSU slice coordinates their requests, handling both uniform and divergent memory accesses. This makes it possible to efficiently support coalesced accesses, where multiple threads access contiguous memory locations, while still correctly servicing irregular or scattered access patterns.

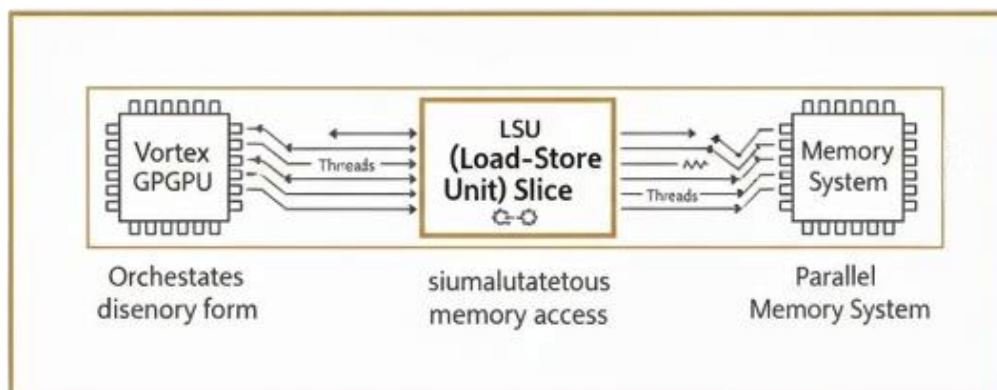


Figure 12,LSU Slice act as Bridge

Role Architecture

Architecturally, the LSU slice acts as the critical interface between the compute pipeline and the memory hierarchy. It translates high-level memory instructions produced by the execution units into low-level memory transactions that can be serviced by the cache and memory system. This includes determining which memory level is being accessed (such as local memory, cache, or global memory) and routing the request accordingly.

The LSU slice also plays a major role in maintaining performance and scalability. By buffering, scheduling, and organizing memory requests from multiple threads, it helps hide memory latency and reduce contention. It interacts closely with the cache hierarchy to exploit data locality and with the memory system to ensure correct ordering and consistency of memory operations. Overall, the LSU slice is essential for sustaining high parallel throughput in the Vortex GPGPU, as it ensures efficient, correct, and scalable memory access across thousands of concurrent threads.

LSU Slice Execution Responsibilities

The LSU slice in the Vortex RISC-V GPGPU is responsible for executing memory-related operations issued by parallel threads, and its execution responsibilities are central to both correctness and performance. One of its primary responsibilities is **address generation**. For each load or store instruction, the LSU slice computes the effective memory address by combining base addresses with per-thread offsets. This must be done independently for each active lane, allowing multiple threads within a warp to access different memory locations in the same cycle.

Another key responsibility is **memory type classification**. After generating addresses, the LSU slice determines which memory space is being accessed, such as local memory, shared memory, cacheable global memory, or other system-level memory regions. This classification is essential because different memory types have different access latencies, coherence properties, and routing paths within the memory hierarchy. Correct classification ensures that each request is forwarded to the appropriate cache or memory unit.

The LSU slice also handles **data format translation**. Memory operations may involve different data widths and types, such as bytes, half-words, words, or vector data. The LSU slice aligns data, performs sign or zero extension when required, and formats the data correctly between the memory system and the execution pipeline. This guarantees that each thread receives data in the expected format, regardless of how it is stored in memory.

The LSU slice manages **parallel lane execution**. Since multiple threads issue memory operations simultaneously, the LSU slice tracks active lanes, handles masked or predicated accesses, and ensures that each lane's request and response are correctly matched. This parallel lane management allows the LSU slice to efficiently support SIMT execution, enabling high-throughput memory access while preserving correct per-thread behavior.

2.9.6 Memory Coalescing, Scheduling, and Response Tracking

The memory subsystem is a primary performance limiter in GPGPU architectures due to the massive number of parallel threads issuing memory operations simultaneously. Under the SIMT execution model, a single load/store instruction may generate multiple memory requests in the same cycle (one per active thread lane). Handling these requests independently leads to redundant transactions, excessive controller overhead, and poor effective bandwidth utilization.

To address these issues, the memory system implements three tightly coupled mechanisms:

1. **Memory Coalescing** to reduce transaction count by merging spatially related thread accesses.
2. **Memory Scheduling** to manage contention and efficiently utilize memory ports and channels.
3. **Response Tracking** (Tagging + Index Buffering) to preserve correctness even when responses return out of order.

Together, these mechanisms improve throughput, reduce stalls, and guarantee correct data delivery to the executing threads.

Design Motivation and Objectives

The purpose of this part is to ensure the memory system can support high concurrency without becoming a bottleneck. The key objectives are:

- **Minimize transaction overhead** by reducing the number of issued memory requests.
- **Maximize effective bandwidth** by issuing wider, cache-line-aligned transactions when possible.
- **Prevent congestion** by buffering and scheduling requests rather than injecting them uncontrollably.
- **Support many outstanding requests** to hide long memory latency and maintain GPU throughput.
- **Guarantee correctness** by reconstructing per-thread responses precisely even under reordering.

These objectives are essential for ASIC-based GPGPU implementations where memory traffic dominates system behavior and determines overall performance.

Memory Coalescing

In a warp execution, multiple lanes often access memory at nearby addresses (e.g., consecutive array elements). If each lane generates an independent memory transaction, the memory system performs repeated overhead work such as request packaging, arbitration, and queue management. This causes:

- Increased memory pipeline pressure
- Higher queue occupancy and backpressure

- Reduced bandwidth efficiency
- Greater latency per useful byte transferred

As the number of active warps increases, this inefficiency becomes a dominant limitation.

Memory coalescing reduces overhead by merging multiple lane requests into fewer wider requests when they map to the same aligned block (typically a cache line). Instead of issuing many narrow transactions, the system issues one wider transaction that covers the necessary region.

This method improves performance by:

- Reducing transaction count
- Increasing the ratio of useful data per memory operation
- Improving cache-line utilization
- Reducing downstream contention in caches and DRAM interfaces

Coalescing is most effective in workloads with spatial locality (vectors, matrices, streaming accesses).

In realistic kernels, not all accesses align perfectly to full words. Some requests may target smaller elements (bytes/halfwords) or unaligned structures. The coalescer handles this using **byte-enable masks** that represent which bytes within the merged block are valid.

For coalesced loads:

- The merged request includes a byte mask indicating which bytes must be returned.
- Returned data is later split and delivered to the correct lanes.

For coalesced stores:

- The coalescer must also pack store-data into the correct byte locations.
- Byte enables ensure only the requested bytes are modified, preventing unintended overwrites.

This mask-based design is important for supporting mixed data types and packed structures while maintaining correctness.

Not all access patterns can be merged efficiently. If thread addresses fall into different cache lines, the coalescer forms multiple transactions rather than forcing incorrect merging. Additionally, if an access crosses a cache-line boundary, the coalescer may split it into two transactions:

- Transaction 1 → first cache line
- Transaction 2 → next cache line

These cases often occur in:

- unaligned vector loads
- structure-of-arrays vs array-of-structures mismatches
- pointer chasing workloads

Even in such scenarios, the coalescer still improves average efficiency whenever locality exists.

Memory Scheduling

Coalescing reduces the number of transactions, but the memory system still faces high request rates because many warps remain active. Since memory resources (ports, banks, channels) are limited, uncontrolled injection leads to:

- Queue buildup and saturation
- Increased contention and service delays
- Backpressure into the LSU/pipeline
- Reduced parallel efficiency across channels

Scheduling is therefore required to regulate traffic and maintain stable throughput.

Scheduler Structure and Responsibilities

The memory scheduler typically implements:

1. **Request queueing** to buffer pending transactions
2. **Arbitration logic** to select which request issues next
3. **Channel/bank mapping** to distribute traffic and avoid hotspots
4. **Flow control** to throttle injection when resources saturate

This allows the memory subsystem to behave as a controlled pipeline rather than a burst-driven system.

GPU execution depends heavily on latency hiding. While one warp stalls on memory, another warp can continue execution. This behavior is only possible if the memory system supports:

- Multiple requests in flight
- Buffered transactions waiting for service
- Out-of-order completion support

The scheduler enables this by maintaining queues and ensuring the pipeline stays active even when some requests experience long DRAM latency.

When multiple memory channels exist, the scheduler can distribute requests across them to increase parallelism. Balanced distribution improves:

- Channel utilization
- Overall throughput
- Reduced waiting time due to contention
- Better system scalability under high traffic

In practice, good scheduling policies prevent cases where one channel becomes overloaded while other channels remain underutilized.

Tag-Based Response Tracking

Memory responses may return in a different order than the issued requests due to:

- Cache hit/miss variations
- Bank conflicts and channel congestion
- Different service times for different memory regions
- Multi-level cache hierarchy effects

Out-of-order completion is expected in high-performance memory systems, especially with parallel channels and buffering.

To preserve correctness, each outgoing transaction is assigned a **unique tag** (transaction identifier). The tag acts as a reference to the original request context. It is used to:

- Identify which response belongs to which request
- Route data back to the correct destination
- Support multiple outstanding requests safely

The tag remains associated with the transaction until the response is fully processed and committed.

Index Buffer for Response Reconstruction

Because coalescing merges multiple lane requests into one wide transaction, the returning data must be split and delivered correctly per lane. The index buffer stores the metadata needed to reconstruct each lane's data response, including:

- Active lane mask (which threads participated)
- Byte offsets within the returned block
- Access width and formatting requirements
- Destination routing information (writeback target)

Without this buffer, the system would lose the mapping between the merged transaction and the original per-thread requests.

When a response returns:

1. The tag identifies the corresponding index-buffer entry.
2. The stored metadata is retrieved.
3. Each lane extracts its correct data slice from the returned block.
4. Alignment and formatting are applied (shift, mask, sign/zero extension).
5. Results are routed back to the pipeline/writeback stage.
6. The index-buffer entry is released for reuse.

This process ensures full correctness even with merged requests and out-of-order completion.

Summary and Contribution

This section presents the core mechanisms that enable a high-throughput memory subsystem suitable for a GPGPU ASIC implementation:

- **Memory coalescing** improves bandwidth efficiency by reducing redundant per-thread transactions and forming cache-line-based requests.
- **Memory scheduling** increases throughput and prevents congestion by buffering and distributing memory traffic across available resources.
- **Tagging and index buffering** guarantee correctness by supporting out-of-order completion and reconstructing accurate per-thread responses.

Together, these components form a scalable memory-access pipeline that sustains parallel execution, hides latency, and maintains correct behavior under heavy memory workload conditions.

2.10 SOFTWARE STACK

The software stack is what allows programmers to write high-level GPU applications (like OpenCL kernels) that get compiled and executed on the Vortex hardware. It connects the application layer, compiler, runtime, and hardware layers into one coherent system.

2.10.1 Compilation Flow

When you write a GPU program (like in OpenCL or CUDA), you’re writing two programs inside the same file:

1. Host code → runs on the CPU (host processor)
2. Device code (kernel) → runs on the Vortex GPU (accelerator)

Step 1. Compilation Separation

You compile the host part and the device part separately.

They work together — the host is like a manager that prepares data and sends “jobs” (kernels) to the GPU to execute in parallel.

Table 6,Compilation Separation

Code Part	Compiler	Target	Output
Host code	Native GCC/Clang	Host CPU (e.g., x86 or ARM)	ELF executable
Device kernel	RISC-V GCC toolchain	Vortex GPU cores	<code>.vxbn</code> or <code>.elf</code> binary

- The Vortex compiler toolchain (riscv32-unknown-elf-gcc) compiles the kernel into RISC-V machine code.
- The host compiler (normal GCC) compiles your CPU-side code that uses libvortex API calls.

Step 2. Host Program Loads the Kernel

At runtime:

- The host program uses the Vortex Runtime Library (libvortex) to:
 - o Load the precompiled RISC-V kernel binary .
 - o Allocate memory on the GPU
 - o Transfer input data from host → GPU memory
 - o Instruct the GPU to start execution

Step 3. GPU Executes the Kernel

- The Vortex hardware executes the kernel instructions on its RISC-V SIMT cores.
- Each core runs multiple warps → each warp runs multiple threads.
- All threads execute the same kernel but on different data elements.

Step 4. Host Waits and Retrieves Results

- The host waits for completion (using synchronization in the runtime).

- Once GPU finishes, results are copied back to host memory.
- Host code continues execution normally.

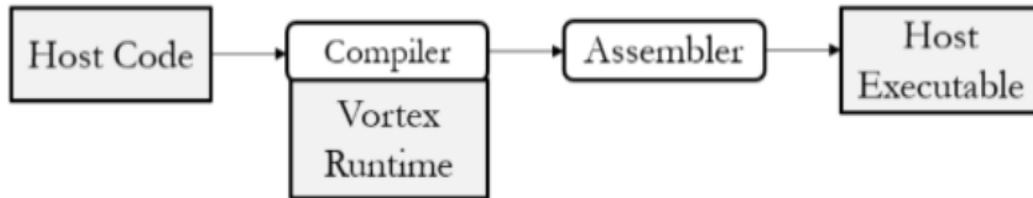


Figure 13, Host completion Flow

2.10.2 GPU Compilation Flow

The flow goes through the following steps:

1. Kernel Code Development:

- Kernel code is written in OpenCL or CUDA.
- It can be written in C++ by adding Vortex custom library.

2. Front-end Compilation (PoCL):

- The kernel code is processed by a front-end compiler.
- The front-end compiler generates Intermediate Representation (IR) from the kernel code, which can help improve optimization.
- Uses built-in libraries for math functions.

3. Back-end Compilation (LLVM):

- The IR is passed to a back-end compiler.
- The back-end compiler generates the kernel executable.
- Uses Vortex Kernel Library for accessing runtime information and controlling program flow (e.g., vx spawn tasks & vx num threads).

- Uses Vortex ISA which includes custom RISC-V instructions for the Vortex GPGPU.

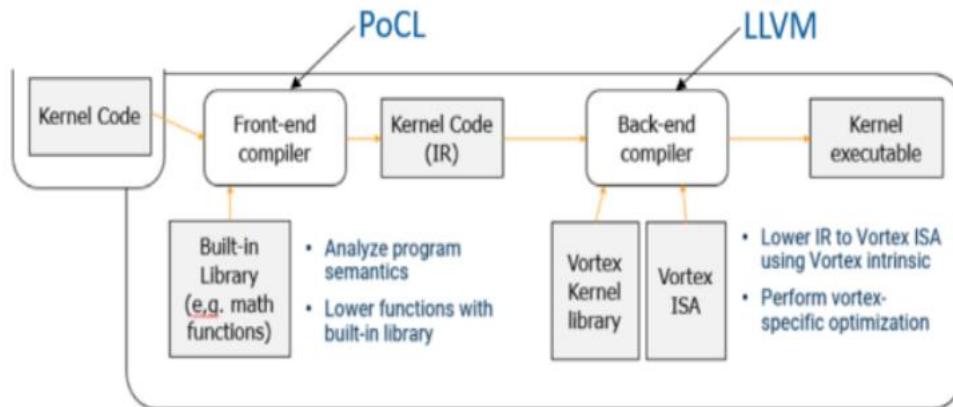


Figure 14, GPU compilation flow

2.11 CONFIGURATION AND INSTRUCTION SET

RISC-V instructions are composed of several fields, each serving a specific purpose. These fields define the operation to be performed, the source and destination registers, and any immediate values required for the instruction.

Table 7, Instruction set of the RISC-V Gpu

Field	Bit Width	Purpose
Opcode	7	Specifies the operation type and determines the instruction format.
rd	5	Destination register that stores the result of the operation.
funct3	3	Encodes the operation's sub-type within the opcode. It refines the instruction's functionality.
rs1	5	First source register used in the operation.
rs2	5	Second source register used in two-operand operations.
funct7	7	Provides additional differentiation for the instruction, particularly for operations like shifts and arithmetic.

Immediate	Variable	Encodes constants or offsets required for specific instructions. Immediate values are always sign-extended.
------------------	----------	--

2.11.1 Base ISA

The RISC-V base ISA defines four primary instruction formats (R, I, S, U) and two additional formats (B, J), each tailored to different immediate encoding needs. All base instructions are 32 bits wide and must be aligned on four-byte boundaries in memory; taken branches or jumps that target misaligned addresses raise misalignment exceptions. Extensions such as the compressed (C) extension introduce 16-bit instructions, enabling two-byte alignment and higher code density. Immediate fields are always sign-extended in hardware to simplify implementation and maintain consistent behavior across formats. The source and destination register fields (rs1, rs2, rd) occupy fixed bit positions in all formats, which reduces decode complexity, while the immediate fields vary by format but are carefully packed and aligned to support efficient extraction and sign extension.

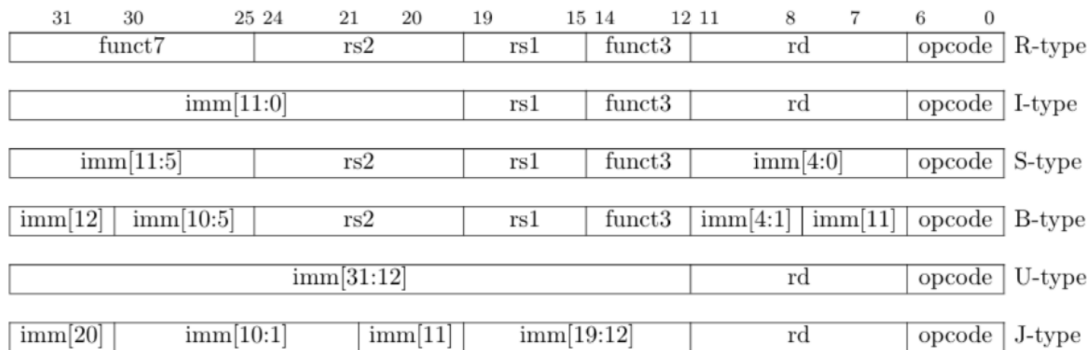


Figure 15, RISC-V Base Instruction Formats

- **R-Type**
 - Used for register–register operations.
 - Specifies two source registers and one destination register.
- **I-Type**
 - Used for operations involving an immediate value.

- Combines one source register with an immediate to produce a result.
- **S-Type**
 - Used for store operations to memory.
 - Writes a source register to memory using an immediate offset as the address displacement.
- **U-Type**
 - Used for upper-immediate operations.
 - Loads a large immediate value into the upper bits of a register.
- **B-Type**
 - Used for conditional branch operations.
 - Compares two source registers and updates the PC to a branch target if the condition is satisfied.

3 ASIC IMPLEMENTATION

3.1 FPGA-TO-ASIC TRANSITION

Moving from FPGA to ASIC represents the path from research prototype to production-grade silicon. This transition involves adapting the RTL to be fully technology-agnostic, replacing FPGA-specific resources (block RAMs, DSP slices) with ASIC standard cells and SRAM macros, and running a complete ASIC flow including synthesis, place-and-route, timing closure, and physical verification.

While FPGA implementations offer flexibility and ease of modification, an ASIC implementation delivers significantly higher performance, lower power consumption, and better area efficiency, at the cost of higher non-recurring engineering (NRE) and reduced post-fabrication flexibility.

3.2 SFU

3.2.1 SFU Synthesis Methodology

This section describes the synthesis flow used to implement the **Special Function Unit (SFU)** of the Vortex GPU using a Synopsys Design Compiler ASIC synthesis tool. The objective of this process is to transform the SFU Register-Transfer Level (RTL) description into an optimized, gate-level netlist while meeting timing, area, and power constraints.

3.2.2 Top-Level Design Definition and Formality Setup

The synthesis process targets the module `VX_sfu_unit`, which represents the top-level SFU block. This module is explicitly declared as the design entry point to ensure that all dependent submodules are correctly elaborated and synthesized. To support formal equivalence checking between the RTL and synthesized netlist, a Synopsys Formality setup file (SVF) is generated during synthesis. This file captures synthesis transformations and enables post-synthesis functional verification.

3.2.3 Library and Technology Configuration

The synthesis flow is configured to use a 7 nm standard-cell technology library, with consistent timing libraries specified for slow-slow (SS), typical-typical (TT), and fast-fast (FF) corners.

These libraries are assigned to both the `target_library` and `link_library`, ensuring that all logic is mapped exclusively to valid standard cells available in the selected process technology.

Multiple search paths are defined to locate RTL source files, interface definitions, and shared package files. This organization ensures modular compilation and allows the synthesis tool to resolve all dependencies across the SFU design hierarchy.

3.2.4 RTL Analysis and Elaboration

RTL compilation follows a structured order to respect SystemVerilog dependencies. First, global package files defining shared types, parameters, and constants are analyzed. Next, interface definitions are compiled, followed by all SFU submodules. The design is then elaborated with `VX_sfu_unit` as the top-level module, producing a fully resolved design hierarchy.

After elaboration, the design is set as the current working design, and a full linking step is performed. This step resolves all module references and binds the RTL logic to the target technology libraries. A design consistency check is executed to verify correctness before optimization begins.

3.2.5 Path Grouping and Timing Organization

To improve timing analysis clarity and optimization efficiency, paths are grouped into three logical categories:

- Input-to-register paths (INREG)
- Register-to-output paths (REGOUT)
- Input-to-output paths (INOUT)

This grouping helps the synthesis tool apply targeted optimization strategies and allows clearer reporting of timing behavior across different path types.

3.2.6 Timing Constraints Definition

Timing constraints are provided through an external constraint file. A single primary clock is defined with a 100 ns period, representing a 10 MHz operating frequency. The clock is modeled with realistic parameters including setup and hold uncertainty, rise and fall transition times, and zero latency.

To preserve clock integrity during synthesis, the clock network is explicitly marked as non-optimizable using a `dont_touch` constraint. This prevents unintended modifications to the clock structure that could affect timing predictability.

3.2.7 Mapping and Optimization

After synthesis completion, the following deliverables are generated:

- Gate-level Verilog netlist for downstream implementation and simulation
- Design Compiler database (DDC) for incremental synthesis or ECOs
- Standard Delay Format (SDF) file for back-annotated timing simulation
- Synopsys Design Constraints (SDC) file reflecting the applied timing constraints
- These outputs collectively support verification, place-and-route, and signoff stages.

These outputs collectively support verification, place-and-route, and signoff stages.

3.2.8 Reporting and Analysis

3.2.8.1 Clock Report Summary

The clock report confirms that the SFU design operates under a single primary clock (CLK) with a period of 100 ns, corresponding to a target frequency of 10 MHz. The clock waveform is defined with a 50% duty cycle, and the clock network is explicitly marked as `dont_touch`, ensuring that the synthesis tool does not modify or optimize the clock structure. No generated or derived clocks are present in the design, simplifying timing closure and analysis. The absence of clock latency and the use of ideal clock modeling indicate that this synthesis run focuses on functional and logical correctness rather than post-layout clock tree effects.

3.2.8.2 Setup Timing Analysis

Table 8: SFU Setup Timing Summary

Metric	Value
Critical Path Delay	0.60 ns
Required Time	99.75 ns
Setup Slack	+99.15 ns
Timing Status	MET
Critical Path Location	Write Control Unit (Elastic Buffer Logic)

The setup timing report shows that the SFU comfortably meets setup timing, with an extremely large positive slack. This confirms that the design operates well below the imposed timing limits, which is expected given the low target frequency. The critical path consists primarily of simple combinational logic and register stages, demonstrating that no deep logic chains exist in the SFU datapath.

3.2.8.3 Hold Timing Analysis

Table 9: SFU Hold Timing Summary

Metric	Value
Worst Hold Slack	−0.02 ns
Timing Status	VIOLATED
Violating Module	CSR Unit
Path Type	Register-to-Register
Violation Severity	Minor

A small hold-time violation is observed on a fast internal path within the CSR unit. Such violations are common when ideal clocks and zero clock latency are used during synthesis. The small magnitude of the violation indicates that it can be resolved easily during place-and-route through buffer insertion or clock skew adjustment, and it does not pose a functional risk at this stage.

3.2.9 Area Utilization Analysis

Power Consumption Analysis

Table 10: SFU Overall Area Summary

Metric	Value
Total Cell Area	~1239.7 μm^2
Combinational Cells	4537
Sequential Cells	1907
Hard Macros	None
Design Style	Fully Standard-Cell

Table 11: SFU Area Distribution by Submodule

Submodule	Area Contribution
Dispatch Unit	~45.6%
CSR Unit	~25.5%
Gather Unit	~16.9%
Other Units	~12.0%

The area results show that control and buffering logic dominate the SFU design. The dispatch unit occupies the largest area due to arbitration, buffering, and control signal generation. The

CSR unit also contributes significantly because of register storage and control paths. No macros or black-box components are present, ensuring a fully synthesizable and portable design.

3.3 FPU

This part document the RTL-to-ASIC synthesis phase of the `VX_fpu_unit` floating-point execution unit for the Vortex GPU project. The FPU was mapped to TSMC 7-nanometer standard cells using Synopsys Design Compiler, achieving timing closure at 10 ns (100 MHz) with 6.69 ns positive slack. Total design area is 385.75 μm^2 with 59.71 μW dynamic power consumption . This represents one phase of a larger ASIC implementation project; formal verification and physical design phases are planned for future work.

3.3.1 Design overview

The `VX_fpu_unit` is a comprehensive floating-point datapath supporting:

- Multiply-accumulate (FMA) operations
- Single-precision type conversions (float $\leftrightarrow$ int)
- IEEE 754 special value handling and classification •
- Multiple rounding modes

The design comprises eight specialized modules (`VX_fpu_dsp`, `VX_fcvt_unit`, `VX_fncp_unit`, `VX_fp_classifier`, `VX_fp_rounding`, `VX_pipe_register`, `VX_lzc`, `VX_shift_register`), all written in standard SystemVerilog without vendor-specific primitives.

3.3.2 Synthesis Constraints

Table 12, Synthesis Constraints

Constraint	Value
Clock Period	10 ns (100 MHz)
Technology	TSMC 7-nm standard cells
Supply Voltage	0.75 V (typical)
Temperature	25°C
Pipeline Depth	5-8 stages

4 VERIFICATION

4.1 HIERARCHICAL VERIFICATION STRATEGY

The "Divide and Conquer" Approach To address the verification complexity of the Vortex GPGPU, a hierarchical "Divide and Conquer" strategy was adopted. The top-level Vortex core is a massive, deeply pipelined architecture; simulating the full system to detect simple logic errors is computationally expensive and inefficient, often requiring minutes or hours per iteration.

To mitigate this, the verification flow decomposes the execution stage into isolated units specifically the Arithmetic Logic Unit (ALU), Floating Point Unit (FPU), Load-Store Unit (LSU), and Special Functions Unit (SFU). This isolation offers three distinct advantages:

- **Complexity Reduction:** It focuses verification strictly on the logic functionality required for the ASIC conversion, eliminating the overhead of full-system interconnects.
- **Fault Isolation:** By decoupling execution units from the top-level hierarchy, the environment ensures that detected failures are genuine logic bugs within the unit, rather than artifacts of system-level communication or bus contention.
- **Parallelism:** This decomposition allows multiple verification engineers to validate different architectural blocks simultaneously without resource blocking

4.2 CHALLENGES AND AUTOMATION

A significant challenge encountered during unit isolation was the management of code dependencies, often referred to as "Dependency Hell". The Vortex Register Transfer Level (RTL) is highly interconnected; extracting a single unit (e.g., FPU_DSP) breaks links to numerous shared packages, headers, and definition files scattered across the repository. Manual tracking of these dependencies initially resulted in unstable simulation environments and frequent build failures.

To resolve this, an automated build infrastructure was developed. A custom, modular Makefile was engineered to automatically identify, aggregate, and resolve the precise dependencies required for each specific unit's testbench. This automation transformed the setup process into a

reliable, one-click operation, ensuring a stable simulation environment that mirrors the top-level architecture.

4.3 TESTBENCH ARCHITECTURE AND IMPLEMENTATION

4.3.1 Self-Checking Testbench Architecture

The verification environment relies on a structured, self-checking testbench architecture designed to compare the Design Under Test (DUT) against a high-level behavioral model. As illustrated in the system block diagram, the architecture consists of three primary components:

- **Stimulus Generator (Driver):** This component is responsible for generating input vectors. It employs a hybrid approach using both constrained randomizations to cover a wide range of input values and directed corner-case vectors to target known coverage holes.
- **Golden Reference Model:** Parallel to the DUT, the input stimulus is fed into a trusted C++ behavioral model. This model serves as the "ground truth," calculating the expected output based on IEEE-754 standards for floating-point operations or standard arithmetic logic.
- **Scoreboard:** The scoreboard acts as the automated checker. It captures the RTL output from the DUT and the predicted result from the Golden Model cycle-by-cycle, flagging any mismatches immediately.

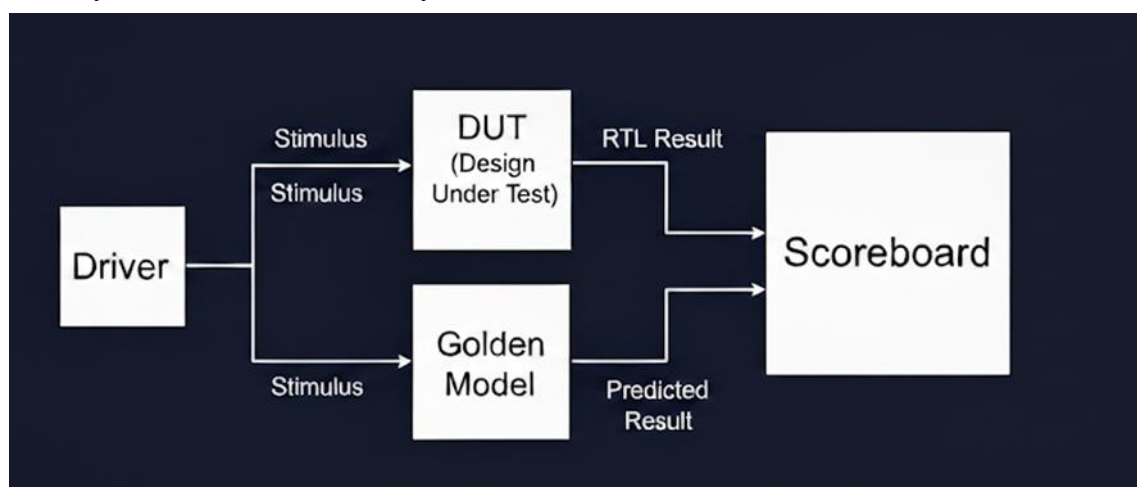


Figure 16, Block Diagram of the Self-Checking Testbench Architecture

4.3.2 System-Level Sanity Checks

While unit-level verification isolates logical bugs within specific execution blocks (like the FPU or ALU), it cannot detect system-level integration issues such as memory mapping errors, bus protocol violations, or synchronization failures between the host processor and the accelerator. To address this, a "System-Level Sanity Check" methodology was implemented. The objective of this phase is to validate the full hardware-software stack, ensuring that the Vortex GPGPU can correctly fetch instructions, execute a kernel, and write valid results back to memory.

4.3.3 OpenCL Verification Flow

The sanity check utilizes an OpenCL-based verification flow, which splits the workload into two distinct domains: the **Host (Software)** and the **Device (Hardware Kernel)**.

Host Code (The Driver): The host application acts as the test manager. It is responsible for allocating shared memory, initializing input buffers (Source A and Source B), and configuring the Vortex processor. The host calculates the **Golden Reference** result in software (C++) before the hardware even finishes execution

```
// verify result
std::cout << "verify result" << std::endl;
int errors = 0;
for (uint32_t i = 0; i < num_points; ++i) {
    auto ref = h_src0[i] + h_src1[i];
    auto cur = h_dst[i];
    if (!Comparator<TYPE>::compare(cur, ref, i, errors)) {
        ++errors;
    }
}
```

Figure 17, OpenCL Host Code of the Software-Based Golden Reference Check

Kernel Code (The Hardware Load): The kernel is the actual program running on Vortex RTL. However, this code does not execute in isolation; it serves as the validation point for the entire **Software Stack**. The kernel relies on the **LLVM-based RISC-V compiler** to translate high-level C logic into machine-readable ISA, and the **Runtime Driver** to manage low-level resource mapping and thread spawning. Successful execution proves that the RTL correctly interprets the instruction stream generated by these upper software layers

```

void kernel_body(kernel_arg_t* __UNIFORM__ arg) {
    auto src0_ptr = reinterpret_cast<TYPE*>(arg->src0_addr);
    auto src1_ptr = reinterpret_cast<TYPE*>(arg->src1_addr);
    auto dst_ptr = reinterpret_cast<TYPE*>(arg->dst_addr);

    dst_ptr[blockIdx.x] = src0_ptr[blockIdx.x] + src1_ptr[blockIdx.x];
}

int main() {
    kernel_arg_t* arg = (kernel_arg_t*)csr_read(VX_CSR_MSCRATCH);
    return vx_spawn_threads(1, &arg->num_points, nullptr, (vx_kernel_func_cb)kernel_body, arg);
}

```

Figure 18,OpenCL Kernel Source Code Executed on the Vortex RTL for System Validation

5 VERIFICATION RESULTS

5.1 FUNCTIONAL VERIFICATION

Extensive simulations were conducted on the isolated execution units. Presents the simulation waveforms for the FPU_DSP block. The waveforms demonstrate the correct propagation of signals, where the result bus matches the expected arithmetic output with valid ready_out and valid_out handshakes. Similar verification fidelity was achieved for the FPU_FPNEW unit, ALU unit, LSU unit, and SFU unit confirming compliance with execution stage blocks.

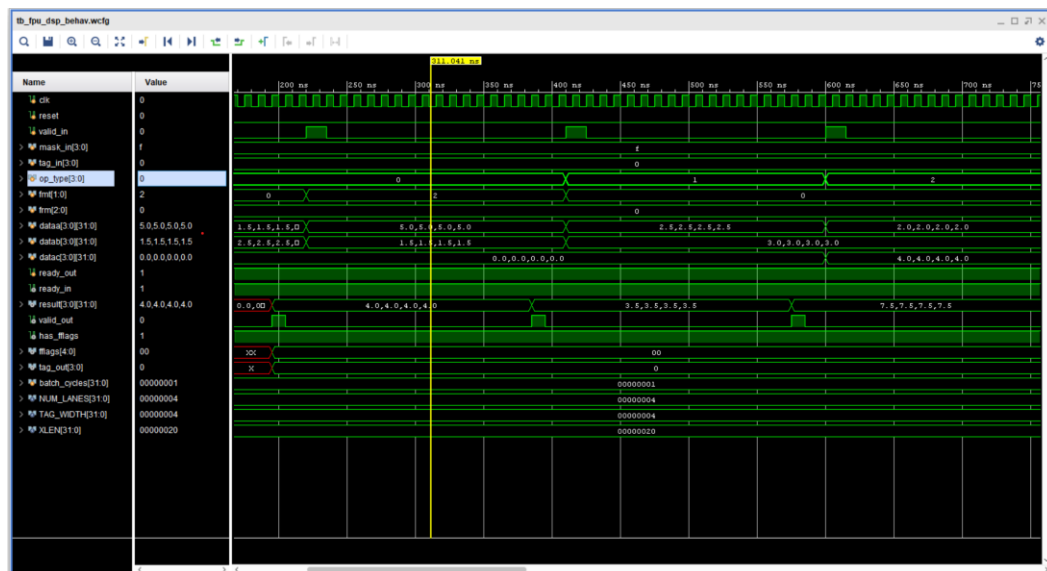


Figure 19,Functional Simulation Waveforms of the FPU_DSP Unit

5.2 REGRESSION TESTING

The robustness of the design was further validated through automated regression scripts. As shown in the terminal output results, the system successfully executed the vecadd kernel, calculating the summation of two vectors each one contain 64 elments. The simulation reported a "PASSED" status with zero errors, confirming that the hardware accurately performed the operations and the host successfully verified the results against the CPU reference.

```

number of points: 64
data type: float
buffer size: 256 bytes
allocate device memory
dev_src0=0x10000
dev_src1=0x10100
dev_dst=0x10200
allocate host buffers
upload source buffer0
upload source buffer1
Upload kernel binary
upload kernel argument
start device
wait for completion
download destination buffer
verify result
cleanup
PERF: core0: instrs=5229, cycles=20824, IPC=0.251105
PERF: core1: instrs=5229, cycles=20825, IPC=0.251092
PERF: instrs=10458, cycles=20825, IPC=0.502185
PASSED!
make: Leaving directory: '/home/al2a2et/Desktop/vector/build/tests/regression/vec

```

Figure 20.: Terminal output showing the successful execution of the vecadd regression test

6 CONCLUSION

This project successfully demonstrated the feasibility of converting a **RISC-V-based GPGPU architecture** from an **FPGA prototype** into an **ASIC-ready design**. Starting from an FPGA-oriented implementation, the architecture was analyzed, refined, and adapted to meet ASIC design constraints, including timing closure, area efficiency, power considerations, and manufacturability.

The transition process highlighted key architectural differences between FPGA and ASIC platforms. FPGA-specific constructs such as vendor IPs, inferred memories, and overly flexible routing structures were replaced with **technology-independent RTL**, **standard-cell-friendly logic**, and **ASIC-compatible memory hierarchies**. Special attention was given to critical GPGPU components such as **SIMT execution units**, **warp schedulers**, **memory arbiters**, and **crossbar interconnects**, ensuring that high parallel throughput was preserved while maintaining reasonable area and power overheads.

From a performance perspective, the ASIC-oriented design benefits from higher achievable clock frequencies, reduced latency, and improved energy efficiency compared to the FPGA implementation. Unlike FPGA designs that rely on reconfigurable logic, the ASIC version exploits optimized standard cells and structured interconnects, making it more suitable for compute-intensive workloads typical of GPGPU applications.

Overall, this work confirms that **RISC-V GPGPU architectures are strong candidates for ASIC implementation**, offering scalability, openness, and design flexibility. The project provides a solid foundation for future enhancements, such as advanced memory systems, improved cache coherence, power optimization techniques, and support for newer parallel programming models. This conversion effort bridges the gap between rapid FPGA prototyping and high-performance ASIC deployment, reinforcing the practicality of open-source GPGPU designs in real silicon.

7 FUTURE WORK

7.1 VERIFICATION

The verification roadmap for the upcoming phase is aligned with the ASIC implementation flow, shifting focus from functional correctness to physical implementation verification.

7.1.1 MILESTONE 1: POST-SYNTHESIS VERIFICATION

Following the synthesis of the Vortex core, the primary objective is to verify that the gate-level netlist is logically equivalent to the golden RTL. This phase will involve Logical Equivalence Checking (LEC) to detect any bugs introduced by synthesis optimizations. Additionally, the simulation environment will be updated to support "Black Box" hard macros, which will replace the behavioral memory models used in the FPGA prototype.

7.1.2 MILESTONE 2: GATE-LEVEL SIMULATION (GLS)

To validate the design under realistic physical conditions, Gate-Level Simulation (GLS) will be performed with Standard Delay Format (SDF) annotation. This step serves as a critical "reality check" to detect X-propagation issues caused by uninitialized states and to validate that the design meets timing constraints that static analysis might miss.

7.1.3 MILESTONE 3: OPTIMIZATION AND BENCHMARKING

In the final phase, the verification team will support microarchitectural optimizations. Continuous regression testing will be employed to ensure that logic optimizations do not break existing functionality. Furthermore, performance verification will be conducted by benchmarking cycle counts, quantitatively verifying that the ASIC team's optimizations yield the intended performance gains.

7.2 ASIC

7.2.1 Milestone 1: Full-Chip Synthesis

The first milestone targets full-chip synthesizability of the Vortex architecture, extending beyond the core to include all remaining subsystems. All RTL blocks must be made synthesis-friendly, for example by replacing behavioral memory models with technology-specific macros or black-box instances. In parallel, SDC constraints—such as clock definitions, clock uncertainty, and I/O delay budgets—are finalized, and a complete gate-level netlist is generated for subsequent physical implementation.

7.2.2 Milestone 2: Baseline Physical Implementation

The second milestone focuses on establishing a baseline backend implementation. Floorplanning defines the overall die size, I/O pin placement, and the arrangement of memory macros and major blocks. Automated place-and-route is then executed, including standard-cell placement, clock tree synthesis, and signal routing, with the objective of producing a layout that is clean with respect to DRC (Design Rule Check) and LVS (Layout Versus Schematic) requirements.

7.2.3 Milestone 3: Microarchitectural Optimizations (Opt1 & Opt2)

The third milestone introduces targeted microarchitectural and RTL optimizations driven by analysis of the baseline implementation. Example changes include refinements to the scheduler, cache hierarchy, or other hotspot blocks to improve either performance, power, or area. The optimized design is then re-implemented and its PPA (Power, Performance, Area) quantitatively compared against the baseline results from Milestone 2 to demonstrate measurable improvement.

7.2.4 Milestone 4: Signoff and Documentation

The final milestone ensures the design meets its timing and documentation requirements. Static Timing Analysis (STA) is performed at the target operating frequency (for example, 500 MHz–1 GHz) to verify that all setup and hold constraints are satisfied across relevant corners. The project concludes with a comprehensive report or paper describing the end-to-end ASIC flow,

key design decisions, optimization steps, and the final PPA metrics achieved by the Vortex implementation.

8ARABIC SUMMARY

يهدف هذا المشروع إلى دراسة وتنفيذ عملية تحويل ال RISC-V GPGPU من FPGA إلى ASIC قابل للتصنيع. تم في البداية الاعتماد على نموذج يعمل على FPGA لاختبار الوظائف الأساسية والتأكد من صحة التنفيذ، ثم تمت إعادة تهيئة التصميم ليتوافق مع متطلبات تصميم الدوائر المتكاملة الخاصة بال ASIC.

ركز المشروع على معالجة الفروقات الجوهرية بين ال FPGA وال ASIC حيث تم استبدال العناصر المعتمدة على بنية FPGA مثل الموارد القابلة لإعادة التهيئة وال IPs الخاصة بالمصنّع بتصميم RTL مستقل وقابل للتخليق باستخدام خلايا قياسية. كما تم تحسين الوحدات الأساسية للمعالج الرسومي العام مثل وحدات التنفيذ المتوازي SIMT، مجدول ال Warps، منظومة الذاكرة، الموزعات (Memory Arbiters)، ومفاتيح الربط (Crossbars) لضمان تحقيق أداء مرتفع مع تقليل المساحة واستهلاك الطاقة.

أظهرت نتائج التحويل أن تصميم ال ASIC يوفر تردداً أعلى، كفاءة طاقة أفضل، وزمناً أقل للوصول إلى الذاكرة مقارنة بتنفيذ ال FPGA، مما يجعله أكثر ملاءمة للتطبيقات كثيفة الحسابات مثل المعالجة المتوازية والحوسبة عالية الأداء. كما يبرز المشروع أهمية معمارية ال RISC-V المفتوحة المصدر في دعم تطوير معالجات GPGPU قابلة للتوسع والتخصيص.

بشكل عام، يؤكد هذا المشروع أن الانتقال من FPGA إلى ASIC يمثل خطوة عملية وفعالة لتحويل النماذج البحثية لمعالجات GPGPU المبنية على RISC-V إلى حلول سيليكون حقيقية ذات أداء وكفاءة أعلى، ويمهد الطريق لأعمال مستقبلية تشمل تحسين إدارة الطاقة، تطوير منظومة الذاكرة، ودعم نماذج برمجة متوازية أكثر تقدماً.

9 REFERENCES

Books:

- [1] Tor M Aamodt, Wilson Wai Lun Fung, and Timothy G Rogers. General-Purpose Graphics Processor Architecture. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, 2018.
- [2] John L. Hennessy and David A. Patterson. Computer Architecture: A Quantitative Approach. Morgan Kaufmann, San Francisco, CA, 5 edition, 2011.

Papers Presented at Conferences (published):

- [3] Georgia Tech University. A general-purpose gpu accelerator. <https://github.com/vortexgpgpu/vortex>, 2023..
- [4] S. Dublsh, V. Nagarajan, and N. Topham, “Characterizing memory bottlenecks in GPGPU workloads,” in Proc. 2016 IEEE Int.Symp. Workload Characterization (IISWC), 2016. [Online].